AI-DRIVEN VOICE BASED INTELLIGENT DESKTOP CONTROL SYSTEM

A Project Report Submitted in the Partial Fulfillment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

B. BHAVANI	**(22W61A0506)**
A. HARITHA	**(22W61A0502)**
B.SAI TEJESWARA RAO	**(22W61A0516)**
A. VASAVI	**(23W65A0502)**

Under the Esteemed Guidance of

Mr. CH VENKATA SSP KUMAR

Assistant Professor

Department of Computer Science & Engineering

Sri Sivani College of Engineering

Approved by AICTE and permanently affiliated to JNTUGV,Vizianagaram

Accredited by NAAC, UGC Recognition under 2(f) & 2(B)

NH-16, Chilakapalem Jn., Srikakulam Dist., Andhra Pradesh - 532410

2022-2026

SRI SIVANI COLLEGE OF ENGINEERING

Approved by AICTE and permanently affiliated to JNTUGV,Vizianagaram

Accredited by NAAC, UGC Recognition under 2(f) & 2(B)

NH-16, Chilakapalem Jn., Srikakulam Dist., Andhra Pradesh - 532410

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that this project work entitled **"AI-DRIVEN VOICE BASED INTELLIGENT DESKTOP CONTROL SYSTEM "** is the Bonafide work carried out by **B.BHAVANI (22W61A0506), A. HARITHA (22W61A0502), B. SAI TEJESWARA RAO(22W61A0516), A.VASAVI(23W65A0502)** submitted in partial fulfillment of the requirements for the Award of the degree of Bachelor of Technology in Computer Science and Engineering, during the year 2022-2026.

Signature of the guide

(Mr. CH Venkata SSP Kumar)

Assistant Professor

Head of the Department

(Mr. Y. Jagadeesh Kumar)

Assistant professor

External Examiner

Institute Vision

To be an institute of eminence, to produce highly Skilled, globally competent technocrats.

Institute Mission

- Providing high quality, real world, industry relevant, career oriented Professional education to rural students towards their excellence and growth.
- Serving as a center of technical excellence, creating globally competent, human resources with ethical and moral values.

DEPARTMENT OF CSE

VISION

Strive to be a center of eminence for developing holistic computer science professionals.

MISSION

M1: To contribute to the advancement of knowledge, in both fundamental and applied Areas of computer Science Engineering.

M2: To promote a sense of excitement among the students in research, design, Development and entrepreneurship.

M3: To nurture communication skills, leadership, ethics and entrepreneurship among Students for their sustained growth.

M4: To forge mutually beneficial relationships with industries and governmental entities

to Develop sustainable computing solutions for the benefits of the society.

SRI SIVANI COLLEGE OF ENGINEERING

Approved by AICTE and permanently affiliated to JNTUGV,Vizianagaram

Accredited by NAAC, UGC Recognition under 2(f) & 2(B)

NH-16, Chilakapalem Jn., Srikakulam Dist., Andhra Pradesh - 532410

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DECLARATION

We do hereby declare that the work embodied in this project report entitled " **AI-DRIVEN VOICE BASED INTELLIGENT DESKTOP CONTROL SYSTEM** " is the outcome of genuine research work carried out by us under the direct supervision of **Mr.CH Venkata SSP Kumar** Asst prof, Department of Computer Science Engineering and is submitted by us to Sri Sivani College of Engineering. The work is original and has not been submitted elsewhere for the award of any other degree or diploma.

Date:

Place:

Registration No.s	Name	Signature
22W61A0506	**B. BHAVANI**	
22W61A0502	**A. HARITHA**	
22W61A0516	**B. SAI TEJESWARA RAO**	
23W65A0502	**A. VASAVI**	

ACKNOWLEDGEMENT

It is indeed with a great sense of pleasure and immense sense of gratitude that we acknowledge the help of these individuals. We feel elated in manifesting our sense of gratitude to our guide **Mr. CH Venkata SSP Kumar**, Assistant Professor in the Department of Computer Science and Engineering for his valuable guidance. He has been a constant source of inspiration for us and we are very deeply thankful to him for his support and invaluable advice.

We would like to sincerely thank our Project Coordinator **Mr. K. Venugopal**, Assistant Professor, Department of Computer Science & Engineering, for giving his direct and indirect support throughout this Main Project.

We would like to thank our Head of the Department **Mr. Y. Jagadeesh Kumar**, Assistant Professor & HOD for his constructive criticism throughout our project.

We are highly indebted to Principal **Dr. Y Srinivasa Rao**, for the facilities provided to accomplish this project.

We are extremely grateful to our Departmental staff members, lab technicians and non-teaching staff members for their extreme help throughout our project. Finally, we express our heartfelt thanks to all of our friends who helped us in successful completion of this project.

Project Members

B.BHAVANI (22W61A0506)
A.HARITHA (22W61A0502)
B.SAI TEJESWARA RAO(22W61A0516)
A.VASAVI (23W65A0502)

ABSTRACT

The modern computing environment requires more natural and efficient ways for users to interact with computers beyond the traditional keyboard and mouse. This project presents NEXA, an AI-Driven Voice-Based Intelligent Desktop Control System that allows users to control a Windows desktop using voice commands. The system is developed using Python and integrates speech recognition, natural language processing, and desktop automation. It uses the Google Speech Recognition API for speech-to-text conversion and OpenAI Whisper as an offline fallback to ensure reliability. To improve accuracy, the system applies Voice Activity Detection (VAD) and noise reduction techniques. A hybrid NLP approach combining a machine learning intent classifier and rule-based patterns is used to understand commands. The system supports tasks like opening applications, file management, web search, and system control, providing an efficient and accessible voice-controlled desktop solution.

Keywords: Voice Control, Speech Recognition, Natural Language Processing, Desktop Automation, Intent Detection, PyQt5, Human-Computer Interaction, NEXA.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

The way human beings interact with computers has undergone a remarkable transformation since the earliest days of computing. From punch cards and command line terminals to graphical user interfaces and touchscreens, each generation of computing technology has progressively moved toward more intuitive, natural, and accessible interaction. Today, the next frontier in this evolution is seamless voice-based interaction, allowing users to communicate with their computers using the same natural language they use to communicate with other people.

Voice-based interfaces have already made significant inroads into the consumer electronics space. Virtual assistants such as Amazon Alexa, Apple Siri, Google Assistant, and Microsoft Cortana have demonstrated the commercial viability and user acceptance of speech-driven interaction. However, these systems are primarily designed for general purpose consumer use such as answering questions, playing music, and setting reminders, and do not provide granular, programmable control over a desktop operating system, its file system, running applications, and system level settings.

The gap between consumer voice assistants and professional-grade desktop voice control tools represents a significant research and development opportunity. This project bridges that gap by developing NEXA, a fully functional, AI-driven voice control system specifically designed to control a Windows desktop environment through natural language voice commands. The rise of deep learning and natural language processing has dramatically improved the accuracy of Automatic Speech Recognition (ASR) systems. Libraries and APIs such as OpenAI's Whisper, and Google's Speech Recognition API now make it possible to transcribe speech with impressive accuracy even in moderately noisy environments, without requiring specialized hardware.

1.2 PROBLEM STATEMENT

Conventional Desktop interaction methods, including mouse, keyboard, and touchpad, while effective for the majority of users, present several significant challenges that limit inclusivity, efficiency, and flexibility in a modern computing environment. Accessibility Barriers: Users with motor disabilities, repetitive strain injuries (RSI), or conditions such as Parkinson's disease face considerable difficulty using traditional input devices. Current desktop environments offer limited built-in voice control, and third-party solutions are either proprietary, expensive, or insufficiently accurate. Workflow Inefficiency: Power users who frequently switch between applications, perform complex file management tasks, and conduct web searches must repeatedly context-switch between keyboard and mouse.

Lack of Intelligent Interpretation: Simple keyword-triggered desktop assistants do not understand natural language. If a user says "open the file I was working on" or "close it after I finish," a keyword-only system fails entirely. Absence of Integrated Solutions: No widely available, open-source, Python based tool offers comprehensive desktop control including application management, file operations, system control, web automation, and dictation within a unified framework. Latency and Reliability: Many cloud-based speech recognition systems suffer from latency or require continuous internet connectivity, limiting practical deployment.

1.2 OBJECTIVES

The primary objective of this project is to design and implement a voice-controlled desktop automation system that allows users to operate a Windows computer using natural language speech. The system includes an audio preprocessing pipeline with Voice Activity Detection (VAD) and noise reduction techniques to improve speech recognition accuracy by reducing background noise and enhancing voice clarity. It integrates the Google Speech Recognition API as the main Automatic Speech Recognition (ASR) engine for converting speech into text, while OpenAI's Whisper model is used as a fallback option to ensure the system continues to work even when internet connectivity is unavailable.

Another objective of the project is to develop a hybrid Natural Language Processing (NLP) pipeline that combines a machine learning intent classification model with a rule-based pattern matching system. This approach helps the system accurately understand user commands across more than 15 command categories. These commands include opening and closing applications, managing files and folders, performing web searches, controlling system settings, text dictation, media control, and retrieving real-time information.

In addition, the project aims to provide a user-friendly graphical interface using PyQt5, which includes features such as a real-time audio energy visualizer, chat style interaction panel, and continuous listening mode for better user interaction. The system also uses a SQLite database integrated with SQL Alchemy to manage user authentication, personalized profiles, and command history. Furthermore, the system supports multicommand execution, where a single spoken sentence can be divided into multiple commands and executed simultaneously using a Multitasking Manager, making the interaction faster and more efficient.

1.3 SCOPE OF THE PROJECT

The system is designed primarily for Microsoft Windows operating systems (Windows 10 and above), utilizing Windows-specific system calls, subprocess commands, and PyAutoGUI for desktop automation. The functional scope covers the complete workflow from voice input capture through audio preprocessing, cloud based and offline transcription, intent recognition, entity extraction, context enrichment, and multi-category command execution. The system handles user authentication with personalized profiles stored in a local SQLite database and provides both visual and audio feedback for every executed command. The scope explicitly excludes macOS and Linux support, cloud deployment, email composition, complex word-processing within documents, and voice biometric authentication, all of which are identified as future enhancements in Chapter 10.

1.5 MOTIVATION

The motivation for NEXA arises from converging real-world needs and academic interests. According to the World Health Organization, over one billion people globally live with some form of disability, many of which affect their ability to use conventional computer input devices. A voice-based desktop control system that works reliably and locally on a standard Windows PC can serve as a transformative assistive technology for this demographic. Beyond accessibility, research in human-computer interaction has consistently demonstrated that multi-modal input systems improve task completion speed and reduce cognitive load. By enabling users to speak commands while their hands remain free, NEXA creates a genuinely multi-modal desktop experience. The availability of powerful, easy-to-use Python libraries for speech recognition, NLP, and machine learning has made it feasible for a small team to build a sophisticated voice control system at minimal cost.

1.6 PROJECT OVERVIEW

NEXA consists of eight key high-level components that operate in a coordinated pipeline:

Audio Subsystem: Captures microphone input, reduces noise, and detects voice.

Speech Recognition Subsystem: Transcribes audio to text via Google STT API (primary) and Whisper (fallback).

NLP Subsystem: Detects intent and extracts entities using hybrid ML and rule-based approach.

Command Execution Subsystem: Routes intents to Application Controller, File Operations, System Controller, Web Automation, and Automation Handler.

Context Management: Context Manager and Context Analyzer maintain session state, resolve pronouns, track history, and provide proactive suggestions.

GUI: PyQt5 Main Dashboard with audio visualizer, chat panel, status indicators, and history dock.

Database: SQLite via SQL Alchemy stores user accounts, preferences, and command history.

CHAPTER 2
LITERATURE SURVEY

2.1 REVIEW OF EXISTING RESEARCH

A thorough review of the existing body of research was conducted to understand the current state of the art in voice-based computer interaction, speech recognition technologies, natural language understanding for command systems, and desktop automation. This section systematically reviews the most relevant and influential works.

2.1.1 Virtual Personal Desktop Assistant

AUTHORS: Tejashri Mane, Tejas Adhude, Karishma Bansode, Prathmesh Pimple, Poorvasha Khairnar, Rohan Sarade (2025). There is a need to develop intelligent desktop assistants that can perform tasks through voice commands to enhance user productivity and accessibility. This research presents Zoro, a customizable Python-based virtual assistant that integrates speech recognition, natural language processing (NLP), and text-to-speech technologies for hands-free desktop interaction. The system successfully performs tasks such as answering queries, managing schedules, controlling desktop functions, and retrieving real-time information through API integration for weather and news updates. Desktop voice assistants like Zoro demonstrate significant potential in streamlining daily computing tasks and improving user experience through personalized and context-aware interactions.

2.1.2 The Ultimate Desktop Assistant (Jarvis)

AUTHORS: Vaishnavi Gote, Mayuri Zele, Falguni De (2025). AI-driven desktop assistants are essential for enhancing user productivity through automation and intelligent task execution. This research presents Jarvis, a virtual assistant that integrates natural language processing (NLP), machine learning, and system automation to provide a seamless user experience. The system emphasizes user-centric design principles and aims to bridge the gap between conventional software-based automation and interactive, voice-controlled AI assistance. Jarvis is designed as a personalized desktop assistant tailored to meet users' specific needs, highlighting the

importance of AI-driven assistance in modern computing to streamline tasks and improve user experiences.

2.1.3 A Desktop Voice Assistant for Real-Time Command Processing

AUTHORS:V.Hemalatha,R.Deepakkannan,R. Dineshkumar, M. Periyakandiyammal (2025). Hands-free desktop control systems are essential for enhancing productivity and accessibility in human-computer interaction. This research creates a personal computer voice assistant using Python that processes real-time voice commands to perform tasks such as sending emails, browsing the internet, and controlling system settings. The system integrates speech recognition, NLP, and text-to-speech technologies with a local SQLite database to store user preferences and frequently used commands for personalized responses. Testing achieved approximately 92% accuracy in voice recognition in quiet environments, demonstrating the potential of voice-controlled interfaces for improving desktop computing efficiency.

2.1.4 Voice Recognition System for Desktop Assistant

AUTHORS: Sudhanshu Gonge, Atishay Jain, Rahul Joshi, Deepali Vora, Ketan Kotecha (2022).Voice recognition technology enables natural human-computer interaction by allowing users to perform tasks through spoken commands without manual input. This research presents a comprehensive desktop voice assistant system built using Python that integrates speech recognition, natural language processing, and texttospeech technologies. The system performs a wide range of tasks including social media management, web searching, sending WhatsApp messages, playing media, retrieving system information like IP addresses, and controlling desktop applications through voice commands. By implementing both offline and online speech recognition capabilities using libraries like CMU Sphinx and Google Cloud Speech API, the assistant provides flexibility in different connectivity scenarios.

2.1.5 Human-computer Interaction Based on Speech Recognition

AUTHORS: Ruixin Lu, Renjie Wei, Jian Zhang (2023). Speech recognition technology enables natural and intuitive interaction between humans and computers by converting spoken language into executable commands. This research analyzes the

key technologies, algorithms, and workflows involved in speech-based human-computer interaction systems, including front-end processing, feature extraction using Mel-Frequency Cepstral Coefficients (MFCC), and modeling methods such as Hidden Markov Models (HMM) and Deep Neural Networks (DNN). The study highlights applications in intelligent assistants, smart home systems, and gaming, while addressing challenges related to environmental noise, speech variations, and contextual understanding. Continuous advancements in deep learning and cross-modal fusion are essential for improving the accuracy and responsiveness of speech recognition systems.

2.1.6 Desktop Voice Assistant System with the Help of Natural Language Processing

AUTHORS: Vishal Bisht (2022). There is a need to develop accessible voice assistant systems to help users, including visually impaired individuals, perform daily tasks more conveniently through handsfree desktop interaction. This research presents a desktop voice assistant built using Python that employs natural language processing (NLP) and speech recognition technologies to interpret and execute user commands. The system integrates modules such as pyttsx3 for text-to-speech conversion and Google Speech Recognition API to enable functions like web browsing, media playback, sending messages, and retrieving information from Wikipedia. By focusing on accessibility, the assistant repeats commands audibly to confirm user input, enhancing usability for visually impaired users and demonstrating how voice-driven systems can improve user experience and independence.

2.1.7 PC Voice Navigation using Python

AUTHORS: Perezada Hamid Ahmad, Harshit Gupta, Monal Raj Singh, Shaan Gupta (Galgotias University) (2023). This paper presents a voice navigation system for Linux-based PCs developed using Python programming language. The system utilizes speech recognition libraries (Google Speech Recognition engine) and text-to-speech synthesis (Pyttsx3) to enable hands-free computer operation. The architecture consists of speech input, speech recognition, natural language processing, and command execution modules. The assistant can open applications, navigate file systems, and

perform basic system commands through voice input. However, the system lacks context awareness, has no memory of past commands, offers no personalization capabilities, and is limited to predefined command sets without multitasking support.

2.1.8 Building an Intelligent Voice Assistant Using Open-Source Speech Recognition Systems

AUTHORS: Venkata Baladari (2023) .This research focuses on designing intelligent voice assistants through open-source speech recognition algorithms including Whisper, Deep Speech, and Kaldi. The paper examines system architecture comprising wake word detection, automatic speech recognition, natural language understanding, dialogue management, and text-to-speech synthesis. Key challenges addressed include accuracy variations across accents, background noise interference, computational requirements, privacy concerns, and multilingual support. The study emphasizes deep learning improvements, emotion sensing AI, and contextual memory capabilities. However, the work remains largely theoretical with no desktop control implementation, limited personalization, and absence of multitasking capabilities or biometric security features.

2.1.9 Gypsy: AI-Powered Virtual Assistant for Windows OS

AUTHORS: V.P.T. Madushan, S.V.S. Gunasekara, B. Fernando (CINEC Campus) (2024) .This paper presents Gypsy, a Windows-based virtual assistant developed in Python with comprehensive functionality including online ordering, web searches, application launching, reminders, weather forecasts, email and WhatsApp messaging, mathematical calculations, and news updates. The system uses Google Speech Recognition API, tkinter GUI, MySQL database for user authentication, and neural networks for dynamic response generation. A notable feature is online order placement through e-commerce platforms. The system includes user registration/login for basic personalization. Despite its broad feature set, Gypsy lacks face recognition authentication, context awareness, multitasking support, translation capabilities, and advanced NLP for understanding complex conversational context.

2.1.10 AI Voice Assistant

AUTHORS: Iliyas Khan, Jatin Rajput, Jeyasekar A. (SRM Institute of Science and Technology) (2024). This paper details a cross-platform AI voice assistant featuring face recognition-based authentication as a key innovation. The system integrates speech recognition, natural language processing, text-to-speech synthesis, and computer vision (OpenCV, face recognition library) for secure user verification. It offers dual interfaces: a desktop GUI and a Streamlit-based web interface for cross-device accessibility. Functionalities include web searches, application launching, and WhatsApp messaging. The modular architecture ensures scalability and maintainability. However, the system lacks context awareness and memory of past interactions, offers minimal personalization, cannot handle multitasking, and provides only basic desktop automation without understanding application context or user preferences.

2.2 COMPARISON OF EXISTING APPROACHES

Table 2.1 presents a structured comparison of NEXA against existing voice control and desktop automation systems across key functional and non-functional dimensions. The comparison reveals that no single existing system addresses all the dimensions that NEXA targets simultaneously.

Table 2.1: Comparison of Existing Voice Control Systems

Feature	Dragon NS	Cortana	Alexa	Siri	NEXA
ASR Engine	Proprietary	Cloud	Cloud	Cloud	Google+Whisper
Desktop Control	Full	Partial	Limited	Limited	Full
File Management	Yes	No	No	No	Yes
Multi-Command	No	No	No	No	Yes
Open Source	No	No	No	No	Yes
Context Aware	Partial	No	No	No	Yes
Offline ASR	Yes	No	No	No	Yes (Whisper)
User Profiles	Yes	Partial	Yes	Yes	Yes
Cost	$300+	Free	Device	Device	Free
Personalization	Limited	Limited	Limited	Limited	Full

2.3 LIMITATIONS OF EXISTING SYSTEMS

Based on the literature survey and comparative analysis, the following key limitations characterize existing voice control systems:

Proprietary and Costly: Commercial solutions such as Dragon NaturallySpeaking require expensive licenses, limiting accessibility for individual users and educational institutions.

Cloud Dependency: Consumer virtual assistants require continuous internet connectivity, making them unsuitable for air-gapped or low-connectivity environments.

Narrow Desktop Control: Consumer assistants cannot perform file system operations, execute local application control, or interact with system configurations at a granular level.

Absence of Context Awareness: Most systems process each command in isolation without maintaining conversational context across turns.

No Multi-Command Support: No available consumer voice assistant supports the parallel execution of multiple commands from a single utterance.

Limited Visual Feedback: Existing systems provide minimal visual feedback about their internal processing state.

No Personalization: Most systems offer limited ability to customize behavior based on individual user preferences or historical command patterns.

NEXA directly addresses all seven of these identified limitations through its architecture, technology selection, and feature design, as demonstrated in Chapters 4 through 7.

CHAPTER 3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Type 1 – Basic Keyword Macro Systems

Early voice control implementations were essentially voice-triggered keyboard macro systems that associated spoken words with fixed keystroke sequences. While functional for simple, frequently repeated tasks, these systems suffer from rigid one-to-one mapping, no understanding of natural language variation, no context awareness, and inability to handle dynamic parameters within commands.

Type 2 – Cloud-Based Virtual Assistants

Systems such as Microsoft Cortana, Apple Siri, Google Assistant, and Amazon Alexa represent the current mainstream approach. These systems use cloud-hosted deep neural network ASR engines, but are primarily designed for information retrieval rather than operating system control, require persistent internet connectivity, continuously transmit user audio to external cloud servers raising privacy concerns, and do not maintain user specific command history in a personalized local manner.

Type 3 – Commercial Dictation Software

Dragon NaturallySpeaking is the most capable commercial voice control product available. It offers genuine desktop control capabilities but requires a significant financial investment exceeding USD 300, a CPU-intensive local speech recognition engine, has no multi-command parallel execution capability, offers no extensibility without purchasing an SDK, and provides no modern visual dashboard interface.

3.2 PROPOSED SYSTEM

NEXA, the AI-Driven Voice Based Intelligent Desktop Control System, is proposed as a comprehensive solution that overcomes all limitations identified in existing systems. The proposed system follows a clearly defined eight-step processing pipeline:

Step 1 – Audio Capture: Continuous VoiceThread captures audio from the default microphone in a background QThread.

Step 2 – Preprocessing: Noise Reducer applies spectral subtraction; Voice Activity Detector trims silence.

Step 3 – Transcription: SpeechToTextProcessor submits audio to Google Speech Recognition API; Whisper handles failures.

Step 4 – Intent Detection: IntentDetector uses ML classification (confidence ≥0.6) then falls back to rule-based regex matching.

Step 5 – Entity Extraction: EntityExtractor identifies application names, file names, and system parameters.

Step 6 – Context Enrichment: ContextManager enriches command with session data; ContextAnalyzer resolves pronoun references.

Step 7 – Execution: CommandExecutor routes enriched command to the appropriate action handler.

Step 8 – Response: Text response added to chat panel; TTS audio via pyttsx3; result logged to database.

3.3 ADVANTAGES OF PROPOSED SYSTEM

Hands-Free Operation: Users can perform complete desktop workflows without touching the keyboard or mouse.

Hybrid Robustness: The dual ASR engine (Google + Whisper) and dual NLP engine (ML + rules) design ensures NEXA continues to function even when individual components fail.

Open Source and Extensible: NEXA's modular Python architecture allows developers to add new command categories without modifying the core system pipeline.

Context Awareness: The ContextAnalyzer maintains session state, enabling natural multi-turn interactions and pronoun resolution.

Multi-Command Execution: NEXA can parse complex multi-action commands and execute each sub-command in parallel using Multitasking Manager.

Zero Cost: Built entirely from open-source Python libraries and free API tiers, requiring no software licensing fees.

User Personalization: The SQLite-backed user profile system stores individual preferences and maintains per-user command history.

Accessibility: Enables complete desktop control through voice alone, serving users with motor or visual disabilities.

3.4 FEASIBILITY STUDY

A structured feasibility study was conducted across four dimensions to validate the viability of the NEXA project prior to development.

Technical Feasibility

All required technologies are mature, well-documented, and freely available. Python 3.8+, PyQt5, SpeechRecognition, scikit-learn, PyAutoGUI, SQLAlchemy, and SQLite are all production-quality tools with extensive community support.

Operational Feasibility

The user interface is designed to be intuitive and require minimal training. Users familiar with consumer voice assistants will find the NEXA interaction model immediately familiar.

Economic Feasibility

NEXA is built entirely from open-source components and the Google Speech Recognition API is available at no cost for moderate usage. There are no licensing fees, subscription costs, or hardware procurement requirements.

Schedule Feasibility

Modular architecture allowed parallel development. Audio/speech (3 weeks), NLP (4 weeks), command execution (4 weeks), GUI and database (3 weeks), integration and testing (3 weeks). Total: one academic semester.

Table 3.1: Feasibility Study Summary

Dimension	**Verdict**	**Key Justification**
Technical Feasibility	Feasible	Open-source Python ecosystem, all tools available
Operational Feasibility	Feasible	Intuitive GUI, minimal training required
Economic Feasibility	Feasible	Zero licensing cost, free API
Schedule Feasibility	Feasible	Modular parallel development within one semester

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE DIAGRAM

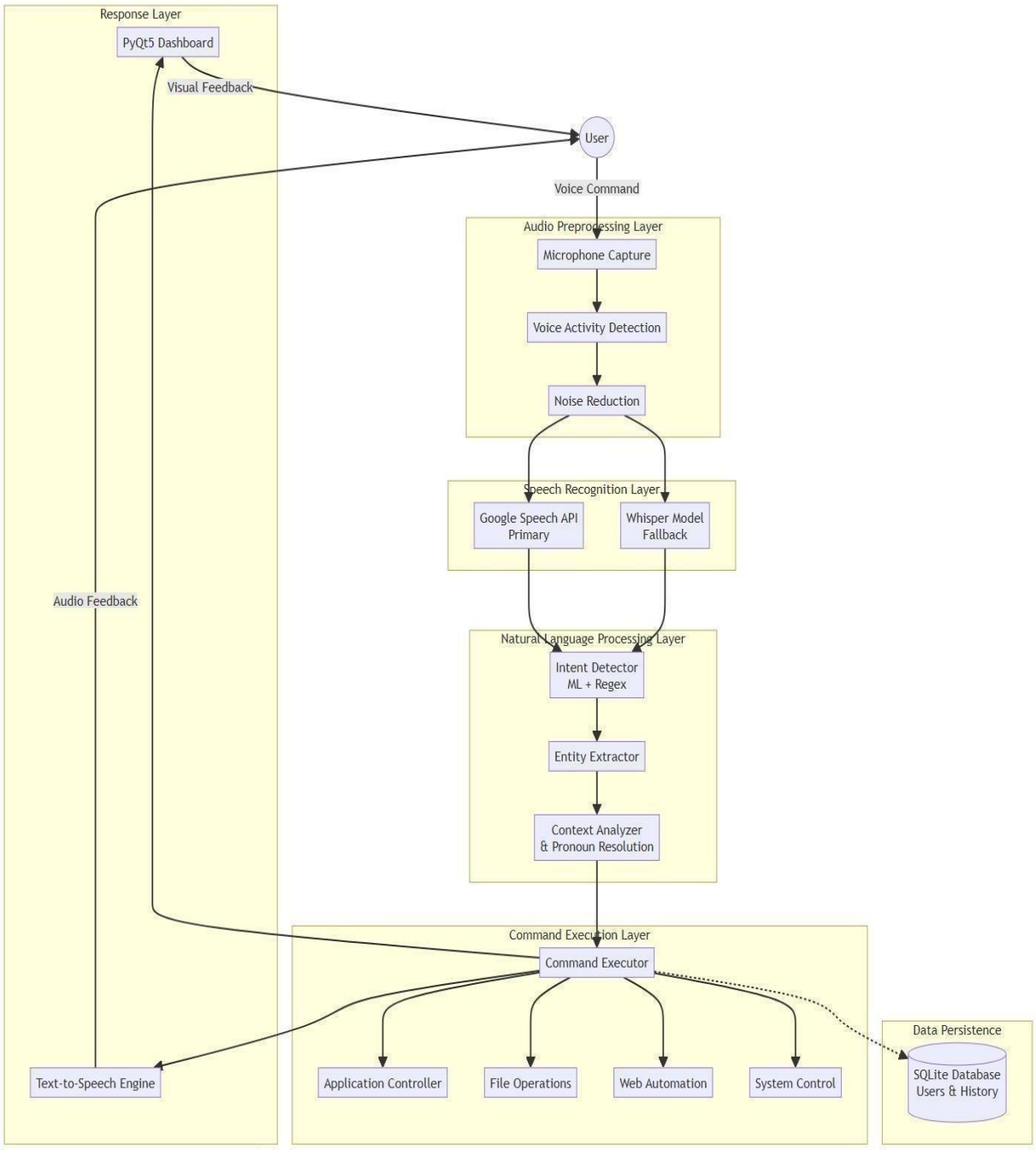

Figure 4.1: System Architecture Diagram of NEXA

The NEXA system is organized into nine distinct layers following a pipeline architecture where each layer receives structured input from the layer above and produces processed output for the layer below. Figure 4.1 illustrates the complete layered architecture. The system follows a clear separation of concerns. Each layer has well-defined responsibilities and communicates with adjacent layers through clearly defined interfaces, ensuring modularity, testability, and maintainability.

4.2 BLOCK DIAGRAM

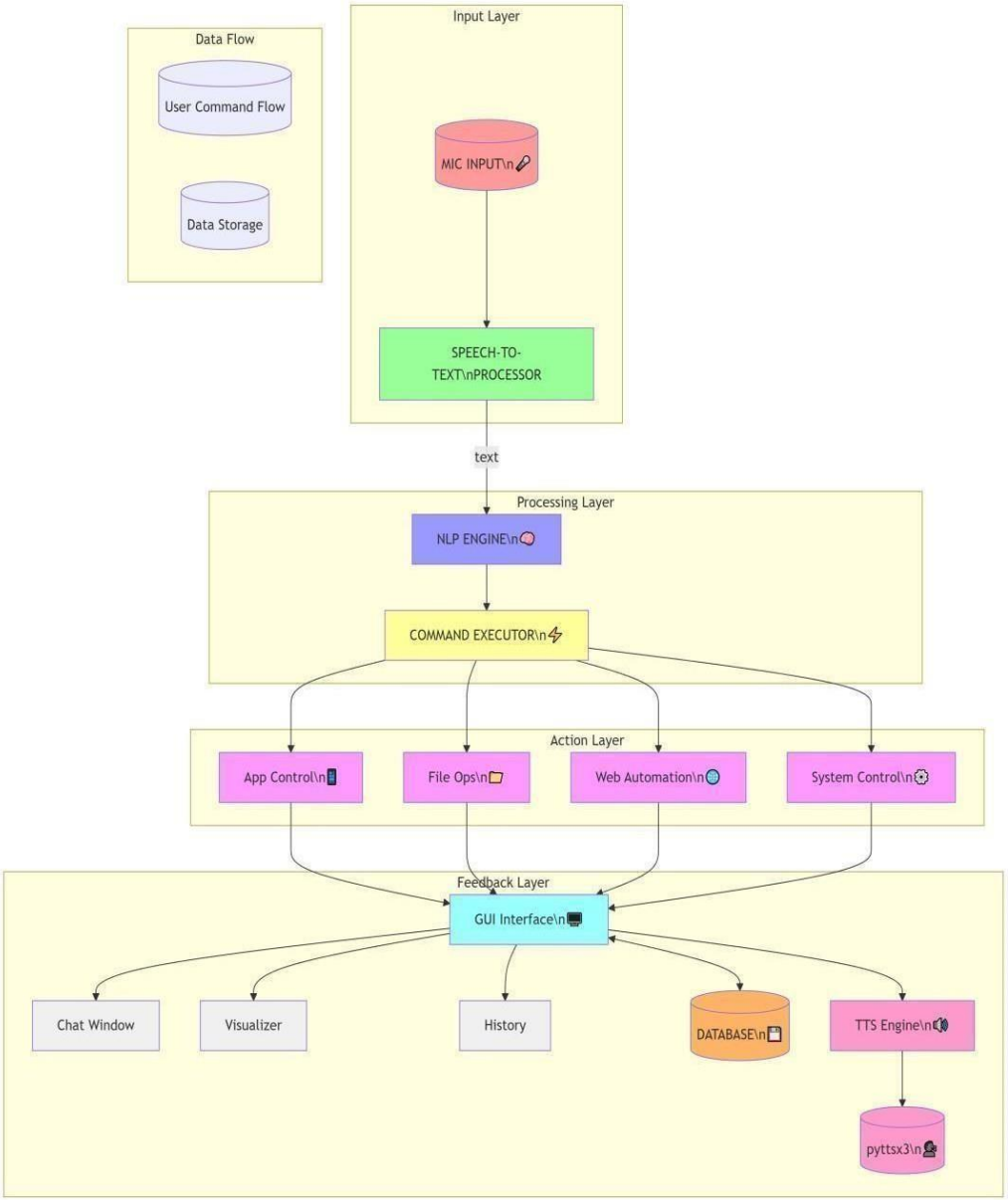

Figure 4.2: Block Diagram of NEXA

The block diagram shows the high-level data flow: audio flows from the microphone to the transcription engine, whose text output feeds the NLP engine. Detected intents drive the command executor, whose results update the GUI and trigger TTS audio feedback. The database layer persists user data and command history bidirectionally with the GUI.

4.3 UML DIAGRAMS

4.3.1 Use Case Diagram

The primary actor is the authenticated User who interacts with NEXA through voice commands captured by the microphone and text commands typed into the input field. Figure 4.3 shows the complete use case model.

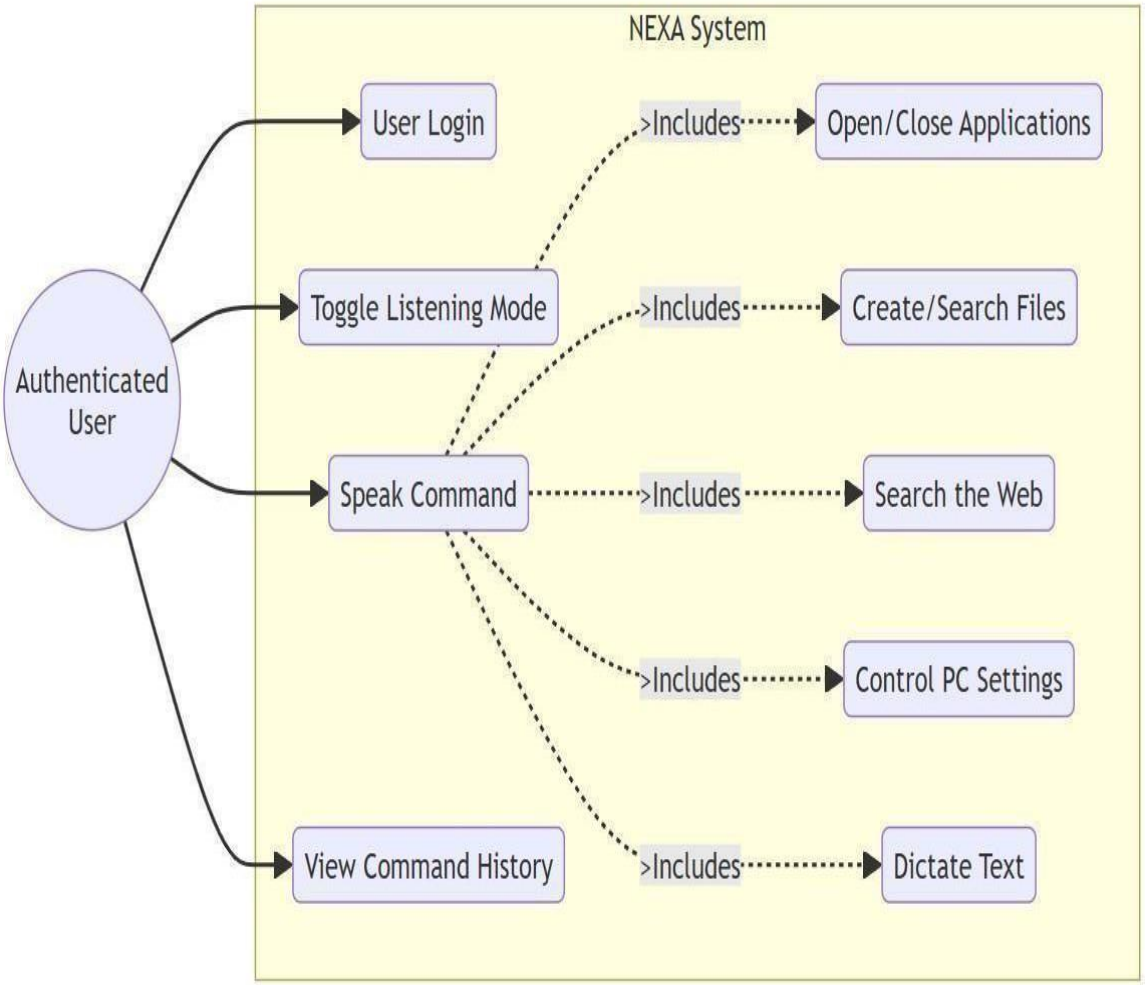

Figure 4.3: Use Case Diagram of NEXA

4.3.2 Class Diagram

The class diagram reveals NEXA's layered dependency structure. VoiceControlEngine acts as the central orchestrator, holding references to all subsystems. The MainDashboard GUI communicates with the engine via Qt signals and slots.

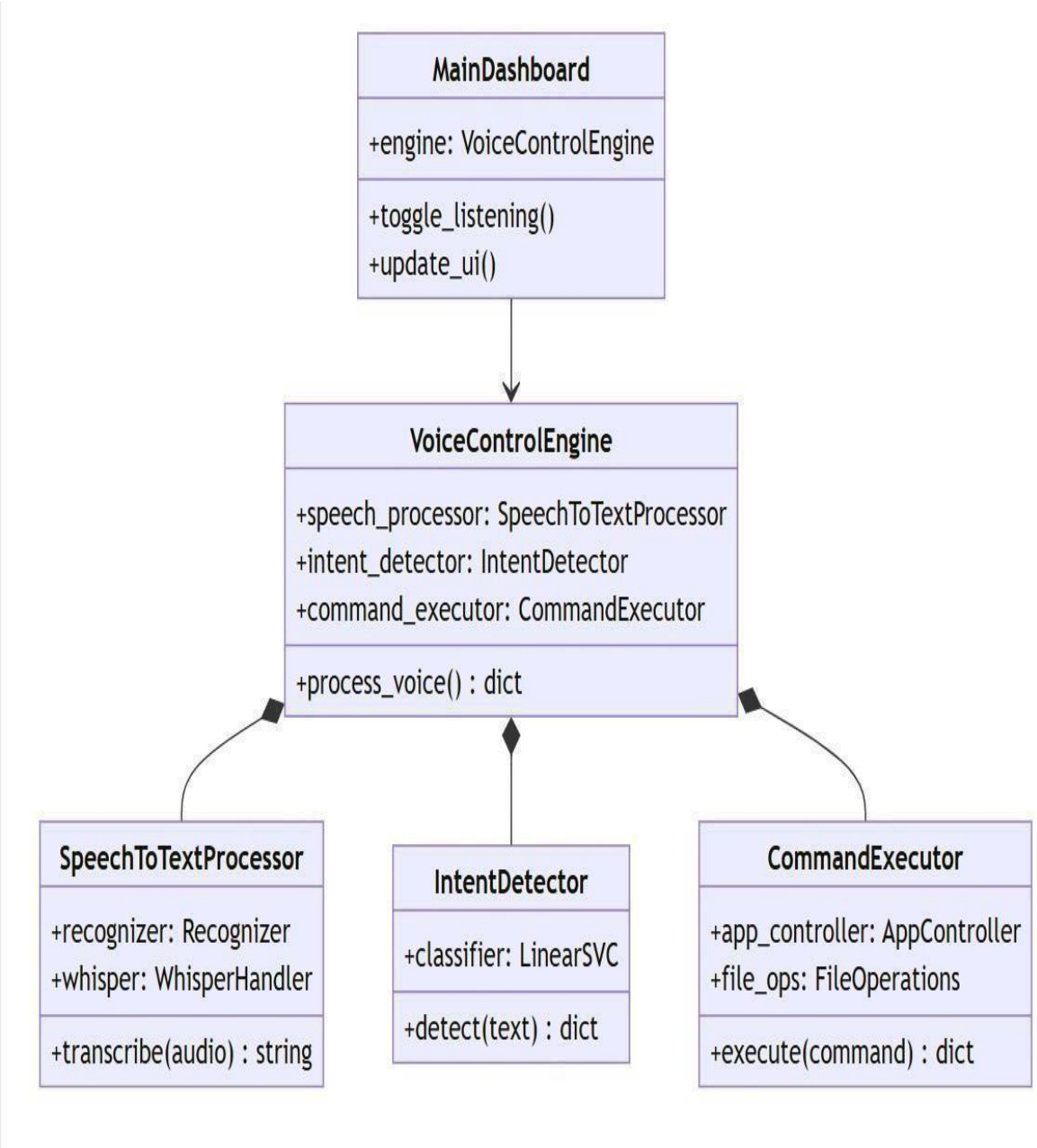

Figure 4.4: Class Diagram of NEXA (Simplified)

4.3.3 Sequence Diagram

The sequence diagram illustrates the message flow during a single voice command cycle. Three worker threads operate concurrently to ensure the GUI remains fully responsive throughout processing.

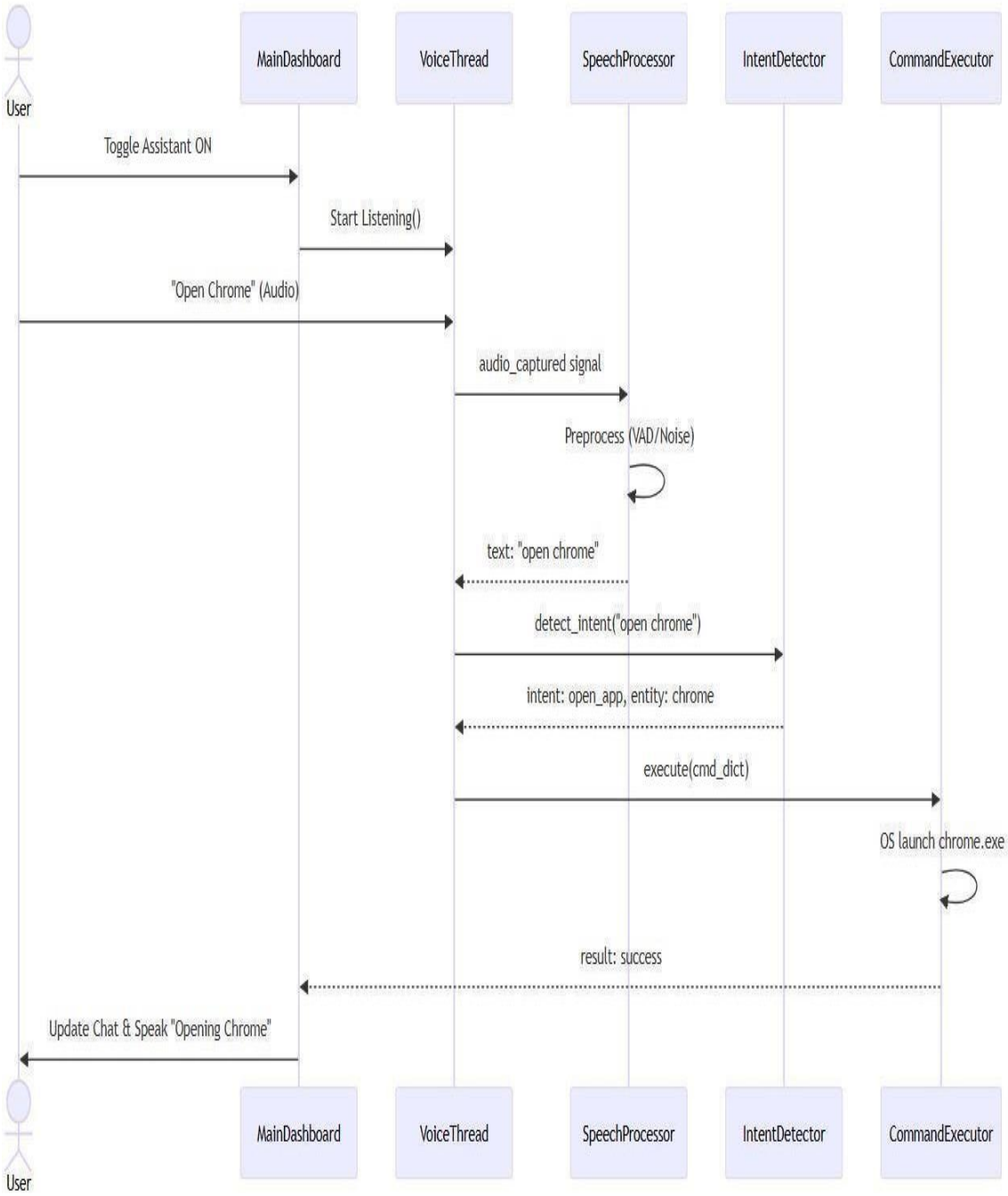

Figure 4.5: Sequence Diagram – Voice Command Processing

4.3.4 Activity Diagram

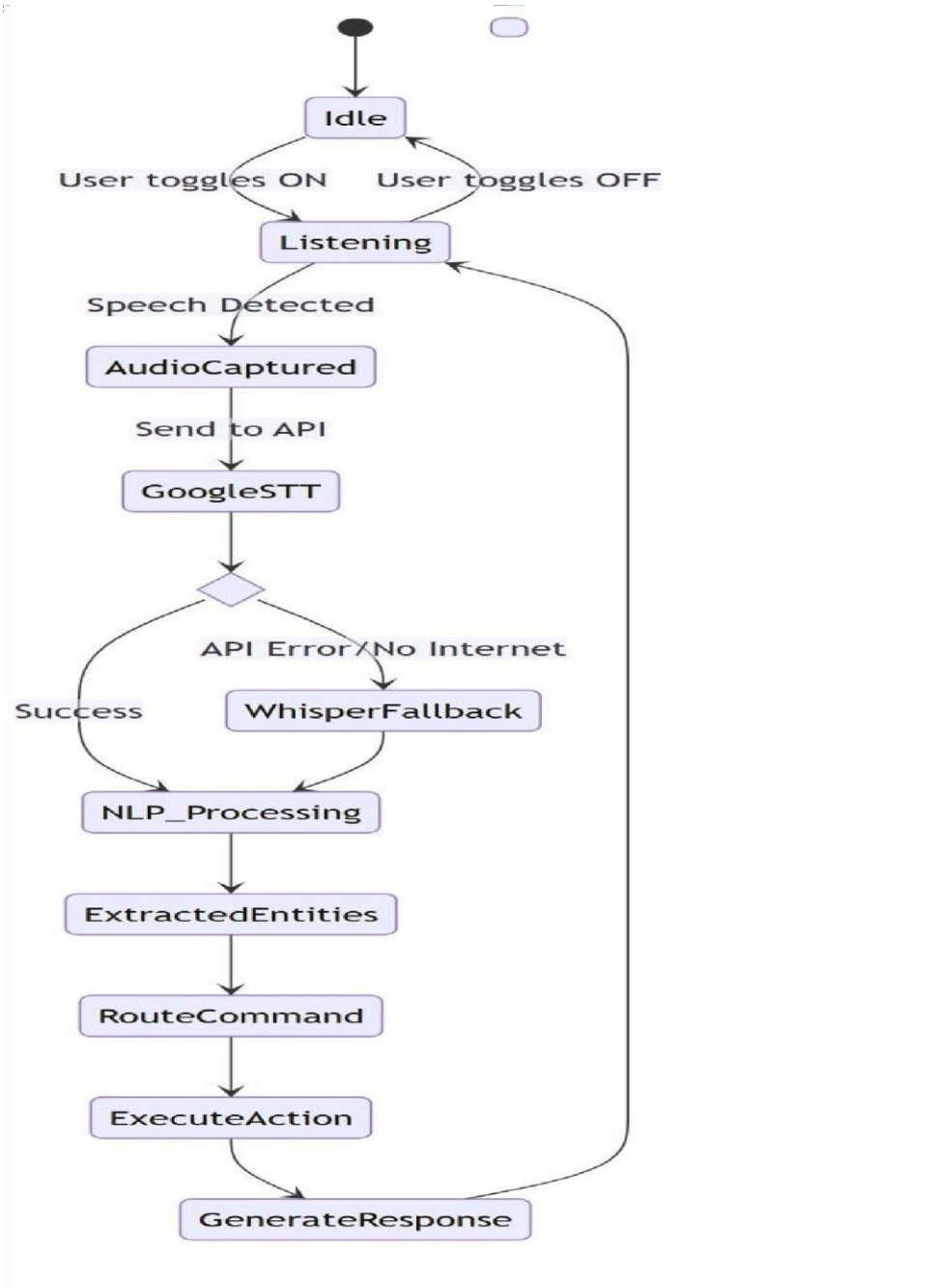

Figure 4.6: Activity Diagram – NEXA Command Flow

The diagram shows a voice command processing pipeline from user toggle to action execution. It features parallel speech recognition paths with GoogleSTT primary and Whisper fallback for offline reliability. The flowchart maps entity extraction and command routing after successful NLP processing. This creates a fault-tolerant system ensuring continuous operation through alternative processing paths.

4.4 DATA FLOW DIAGRAMS

DFD Level 0

The Level 0 DFD presents NEXA as a single process interacting with two external entities: the User and the Google Speech Recognition API. All internal processing is hidden at this level to show the system boundary.

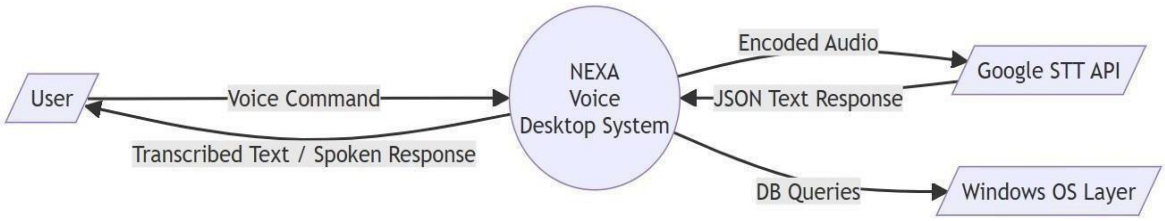

Figure 4.7: DFD Level 0 (Context Diagram)

4.5 DATABASE DESIGN

The database subsystem uses SQLite as the storage engine, accessed via SQLAlchemy ORM. Figure 4.8 shows the database schema with three primary tables.

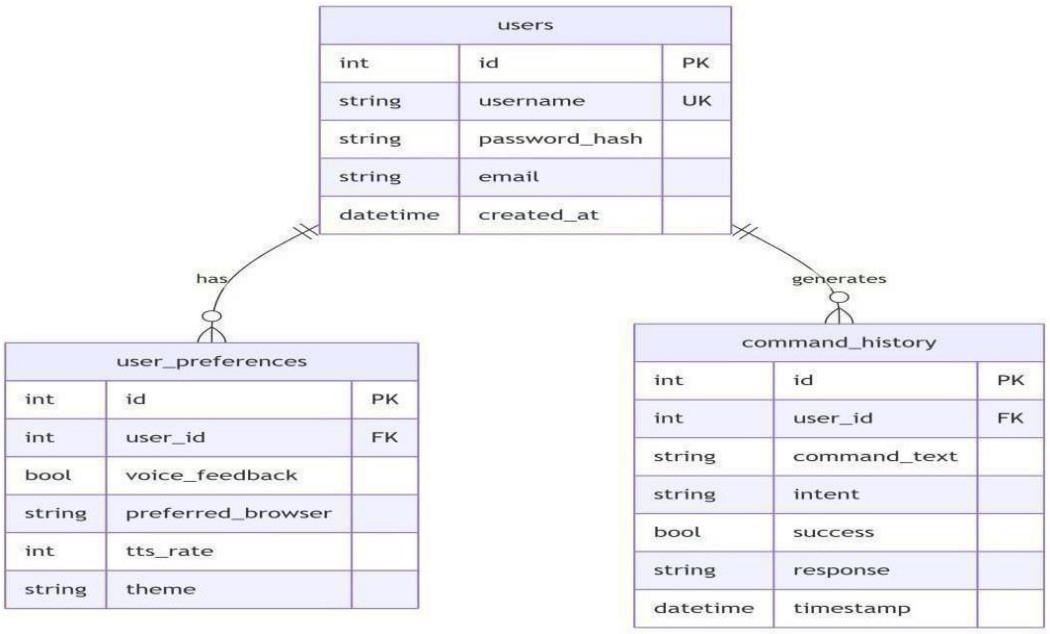

Figure 4.8: Database Schema of NEXA

CHAPTER 5

METHODOLOGY

5.1 PROCESSING PIPELINE OVERVIEW

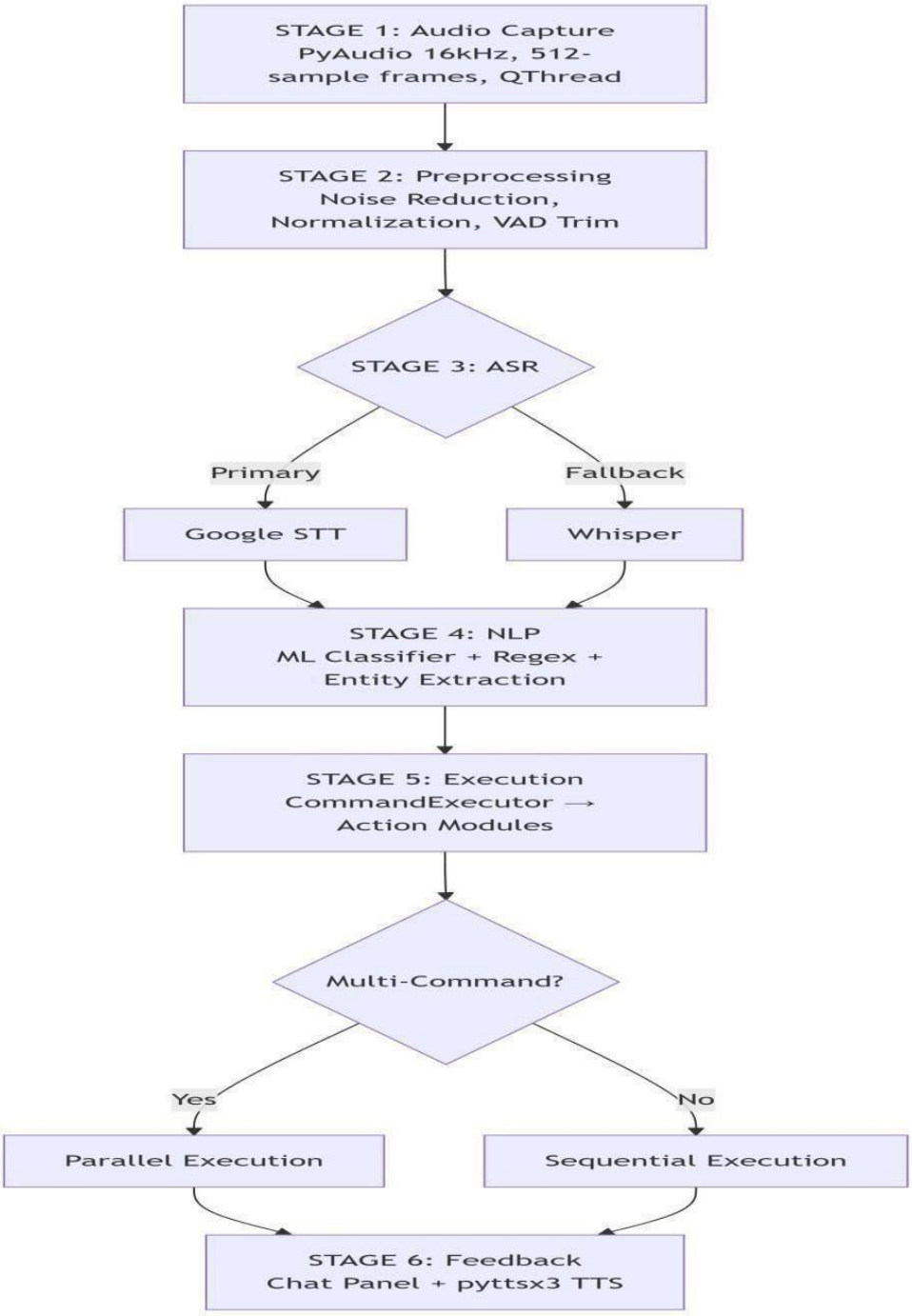

Figure 5.1: End-to-End Processing Pipeline of NEXA

5.2 AUDIO CAPTURE AND PREPROCESSING

Audio capture is handled by the ContinuousVoiceThread running in a background QThread. Before entering the main listening loop, it calls adjust_for_ambient_noise() with a two-second calibration period to set the energy threshold appropriate for the current acoustic environment. The listening loop uses listen() with a one-second timeout and maximum phrase duration of five seconds. When a phrase is detected, the AudioData object is emitted via the audio_captured Qt signal to the TranscriptionWorker queue. This separation ensures the capture thread remains continuously available without being blocked by transcription latency.

Voice Activity Detection, implemented in Voice Activity Detector, computes Root Mean Square (RMS) energy per 512-sample audio frame: energy = sqrt(sum(frame²) / frame_length). Frames with energy below threshold 0.02 are classified as silence. The trim_silence() method returns only the voice-active segment, reducing transcription input and improving response latency. The NoiseReducer applies spectral subtraction to reduce stationary background noise before the audio reaches the ASR engine.

5.3 SPEECH RECOGNITION STAGE

The SpeechToTextProcessor manages the transcription pipeline and supports three input paths:

Path 1 – sr. AudioData: Audio captured by the background listener is submitted directly to Google Speech Recognition API via recognize_google(). This is the primary, lowestlatency path averaging 1.2 seconds end-to-end.

Path 2 – NumPy Array: When audio arrives as a numpy float32 array, noise reduction and VAD trimming are applied, the data is converted to 16-bit PCM bytes, wrapped in sr.AudioData, and submitted to the Google API.

Path 3 – Whisper Fallback: If both Google API paths fail due to UnknownValueError or connectivity issues, WhisperHandler.transcribe() is used for fully offline local transcription. All transcribed text is lowercased and stripped for consistent downstream processing.

5.4 NATURAL LANGUAGE PROCESSING STAGE

5.4.1 Intent Detection

The IntentDetector implements a two-tier detection strategy. Tier 1 uses the IntentClassifier, which loads a pre-trained scikit-learn TF-IDF vectorizer paired with a Linear Support Vector Classifier (LinearSVC). When a text command is received, the classifier predicts the intent label and returns a confidence score. If confidence is ≥0.6, the prediction is immediately accepted.

A domain-specific heuristic override is applied before this threshold check: if the ML model predicts web_search or open_app but the command contains clear filerelated keywords (file, folder, pdf, docx) without operation-creating keywords (create, delete, copy, move), the intent is overridden to file_search, preventing the most common confusion pattern. Tier 2 applies comprehensive regular expression patterns if ML confidence is below 0.6. Examples: open_app matches "open|launch|start|run"; time_date matches "time |date |clock |today"; system_control matches "shutdown|restart|volume|brightness". Rulebased matches receive fixed confidence of 0.7.

5.4.2 Entity Extraction

The EntityExtractor identifies Application Names (chrome, notepad, vlc, word, teams, spotify), File Names and Paths (common extensions pdf, docx, txt, xlsx, mp4), Search Queries (remaining text after command keywords are stripped), and System Parameters (volume levels, brightness percentages).

5.4.3 Context Enrichment and Multi-Command Parsing

The ContextAnalyzer resolves pronoun references. When a command contains "it," "this," or "that," the method checks the ContextManager for the most recently tracked application context. For example, "close it" after opening Chrome resolves to "close chrome" before intent detection. Multi-command parsing splits input text by connector words ("then," "and then," "afterwards") and evaluates whether each resulting part represents an independent command. If they do, each sub-command is processed and executed in parallel by MultitaskingManager using Python threading.

5.5 COMMAND EXECUTION STAGE

The CommandExecutor reads the intent field, calls ActionMapper to translate intent to action string, routes to the appropriate execution method, and returns a standardized result dict {success: bool, message: str}. ApplicationController uses subprocess.Popen and os.startfile to launch applications, maintaining a mapping of common application aliases (e.g., "chrome" -> "chrome.exe"). File Operations implements create_file (), create_directory(), delete_file(), copy_file(), move_file(), rename_file(), search_files() using Python's os and shutil modules. SystemController controls system settings via subprocess and ctypes Windows API calls. WebAutomation opens browser-based searches using Python's webbrowser module for Google, YouTube, Bing, Wikipedia, and Amazon. AutomationHandler uses PyAutoGUI for GUI automation including dictation typing, Ctrl+S for save, and Ctrl+F for inapplication search.

5.6 RESPONSE AND FEEDBACK STAGE

The AssistantVisualizer transitions between four states: IDLE (grey), LISTENING (blue, animated pulse), THINKING (orange), and SPEAKING (green). The AudioEnergyWorker samples the microphone at 512-sample intervals and emits normalized RMS energy [0.0, 1.0] for real-time amplitude visualization in the dashboard. The TextToSpeech module wraps pyttsx3 and launches TTS in a dedicated daemon thread, preventing audio output from blocking the GUI or listening thread. NEXA uses randomized response pools for fillers, confirmations, and follow-ups, ensuring no two consecutive commands receive identical wording and creating a natural conversational experience.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 SOFTWARE REQUIREMENTS

Table 6.1: Software Requirements Specification

Software / Tool	**Version / Details**
Operating System	Windows 10/11 64-bit
Programming Language	Python 3.8+
GUI Framework	PyQt5 5.15.9
Speech Recognition	SpeechRecognition + Google STT API
Text-to-Speech	pyttsx3 2.90
Audio I/O	PyAudio
Machine Learning	scikit-learn (TF-IDF + LinearSVC)
Database ORM	SQLAlchemy 2.0.23 + SQLite
Desktop Automation	PyAutoGUI 0.9.53
Configuration	PyYAML 6.0.1, python-dotenv 1.0.0
Image Processing	Pillow 10.1.0
Audio Processing	NumPy, librosa, soundfile
IDE	Visual Studio Code / PyCharm
Control	Git

6.2 HARDWARE REQUIREMENTS

Table 6.2: Hardware Requirements Specification

Hardware Component	**Minimum Specification**
Processor	Intel Core i3 8th Gen / AMD Ryzen 3 equivalent
RAM	4 GB (8 GB recommended for Whisper model)
Storage	2 GB free disk space
Microphone	USB or 3.5mm microphone input
Internet	Required for primary Google STT API
Display	1366 × 768 resolution or higher
Operating System	Windows 10/11 64-bit

6.3.PYTHON LIBRARIES USED

Table 6.3: Python Libraries Used in NEXA

Library	**Version**	**Purpose**
PyQt5	5.15.9	Main GUI framework
SpeechRecognition	Latest	Google STT API access
pyaudio	Latest	Low-level microphone capture
pyttsx3	2.90	Offline text-to-speech output
numpy	Latest	Audio array operations, RMS energy
librosa	Latest	Audio feature extraction
scikit-learn	Latest	Intent classification (TF-IDF + SVC)
pyautogui	0.9.53	Desktop GUI automation
keyboard	0.13.5	Global keyboard shortcut handling
SQLAlchemy	2.0.23	ORM for SQLite database access
webbrowser	Built-in	Browser-based search automation
subprocess	Built-in	OS application launch and system commands
re	Built-in	Rule-based NLP pattern matching
threading	Built-in	Background thread management
Pillow	10.1.0	Image loading for GUI assets
python-dotenv	1.0.0	Environment variable configuration

6.4 MODULES AND CODE SNIPPETS

6.4.1 Application Entry Point (main.py)

The main.py file defines the ApplicationInitializer class, which performs a six-step startup sequence: environment check, dependency verification, configuration loading, database initialization, ML model verification, and PyQt5 application creation. The robust initialization ensures that missing dependencies or misconfigured paths are caught before the GUI is shown.

Code snippet 6.1: Application Initialization Sequence(main.py)

```
def initialize_all(self):
    """Initialize all components in sequence"""
    if not self.check_environment():
      self.logger.critical("Environment check
failed")
        return False
    if not self.check_dependencies():
        self.logger.critical("Dependency check
failed")
        return False
    if not self.load_configuration():
        self.logger.critical("Configuration loading
        failed")
        return False
    if not self.initialize_database():
        self.logger.critical("Database initialization
failed")
        return False
    self.verify_models()
    if not self.create_application():
        return False

    return True
```

6.4.2 Speech-to-Text Processor (src/speech/speech_to_text.py)

SpeechToTextProcessor manages the transcription pipeline. The recognizer is configured with pause_threshold=0.5 (faster than default 0.8), energy_threshold=300, and dynamic_energy_threshold=True.

Code Snippet: Code Snippet 6.2: Speech Transcription Pipeline

```
def transcribe(self, audio_data, sample_rate=16000):
    """Transcribe audio to text using Google STT or Whisper"""

    if isinstance(audio_data, sr.AudioData):
        try:
            return self.recognizer.recognize_google(audio_data)
        except sr.UnknownValueError:
            return ""
        except Exception as e:
            logger.warning(f"Google STT failed: {e}")

    processed = self.noise_reducer.reduce_noise(audio_data)
    processed = self.vad.trim_silence(processed)

    if len(processed) < 1600:
        return ""

    try:
        audio_int16 = (processed * 32767).astype(np.int16)
        source = sr.AudioData(audio_int16.tobytes(), sample_rate, 2)
        return self.recognizer.recognize_google(source)
    except:
        return self.whisper.transcribe(processed)
```

6.4.3 Voice Activity Detector (src/speech/vad.py)

Code Snippet: Code Snippet 6.3: Voice Activity Detection Module

```
class VoiceActivityDetector:
    def init_(self, threshold=0.02):
        self.threshold = threshold
        self.frame_length = 512

    def detect_voice_activity(self, audio_data):
        frames = self._frame_audio(audio_data)
        energies = [np.sqrt(np.sum(f**2) / len(f)) for f in frames]
        return [e > self.threshold for e in energies], energies

    def trim_silence(self, audio_data):
        voice_activity, _ = self.detect_voice_activity(audio_data)
        voice_frames = [i for i, v in enumerate(voice_activity) if
v]

        if not voice_frames:
            return audio_data

        start = voice_frames[0] * self.frame_length
        end = (voice_frames[-1] + 1) * self.frame_length
        return audio_data[start:end]
```

6.4.4 Hybrid Intent Detector (src/nlp/intent_detector.py)

Code Snippet 6.4: Hybrid Intent Detection

```
def detect(self, text, confidence_threshold=0.6):
    text_lower = text.lower()
    if self.use_ml:
        prediction = self.classifier.predict(text)
        intent = prediction["intent"]
        confidence = prediction["confidence"]
        file_kw = ["file", "folder", "pdf", "docx", "spreadsheet"]
        op_kw = ["create", "delete", "copy", "move"]
        if (intent in ["web_search", "open_app"] or confidence < confidence_threshold) and \
           any(k in text_lower for k in file_kw) and \
           not any(k in text_lower for k in op_kw):
            return {
                "intent": "file_search",
                "confidence": 1.0,
                "source": "heuristic_override"
            }
        if confidence >= confidence_threshold:
            return {
                "intent": intent,
                "confidence": confidence,
                "source": "ml"
            }
    for pat_intent, patterns in self.intent_patterns.items():
        for pattern in patterns:
            if re.search(pattern, text_lower):
                return {
                    "intent": pat_intent,
                    "confidence": 0.7,
                    "source": "rule_based"}
    return {"intent": "unknown", "confidence": 0.0}
```

6.4.5 Main Engine — Orchestrator (src/core/main_engine.py)

Code Snippet 6.5: Single Command Processing Pipeline

```
def _execute_single_command(self, text):
    resolved = self.context_analyzer._resolve_pronouns(text)
    intent_result = self.intent_detector.detect(resolved)
    entity_result = self.entity_extractor.extract_entities(resolved)

    apps = entity_result.get("grouped", {}).get("APP", [])
    if apps:
        self.context_analyzer.update_application_context(apps[0])

    command = intent_result.copy()
    command["text"] = resolved
    command["grouped_entities"] = entity_result.get("grouped",
{})

    enriched = self.context_manager.enrich_intent(command)
    res = self.command_executor.execute(enriched)

    if res.get("success") and res.get("message"):
        self.speak(res["message"])
    else:
        self.speak("Sorry, I could not do that.")

    self.context_manager.add_to_history(resolved, res)
    return res
```

6.4.6 Command Executor (src/actions/command_executor.py)

Code Snippet: Code Snippet 6.6: Command Routing Logic

```
def_execute_internal(self, command):
    intent = command.get("intent", "unknown")
    action = self.action_mapper.map_intent(intent)

    handlers = {
        "open_app": self._execute_open_app,
        "close_app": self._execute_close_app,
        "file_operation": self._execute_file_operation,
        "file_search": self._execute_file_search,
        "search": self._execute_web_search,
        "system_control": self._execute_system_control,
        "dictation": self._execute_dictation,
        "time": self._execute_time,
        "weather": self._execute_weather,
        "music": self._execute_music,
    }
    fn = handlers.get(action)
    if fn:
        return fn(command)
    return {"success": False, "message": "Command not available."}
```

6.4.7 Continuous Voice Thread (src/gui/main_dashboard.py)

Code Snippet 6.7: Background Listening Thread

```
class ContinuousVoiceThread(QThread):
    audio_captured = pyqtSignal(object)

    def run(self):
        self.running = True
        mic = sr.Microphone()

        with mic as source:

self.processor.recognizer.adjust_for_ambient_noise(
                source, duration=2
            )

            while not self._stop_event.is_set():
                try:
                    audio = self.processor.recognizer.listen(
                        source, timeout=1, phrase_time_limit=5
                    )
                    self.audio_captured.emit(audio)

                except sr.WaitTimeoutError:
                    continue  # silent period, keep listening

                except Exception as e:
                    if not self._stop_event.is_set():
                        self.error_occurred.emit(str(e))
```

Table 6.4: Supported Command Categories in NEXA

Category	Example Commands
Application Control	"Open Chrome", "Close VLC", "Launch Notepad"
File Operations	"Create folder projects", "Delete backup file"
File Search	"Find my resume", "Search for PDF files"
Web Search	"Google Python tutorials", "YouTube rock music"
System Control	"Shutdown", "Volume up", "Lock screen"
Time & Date	"What time is it", "What day is today"
Dictation	"Type hello world", "Write this text"
Music Control	"Play rock music", "Next track", "Pause"
Weather	"What is the weather today"
Greetings & Help	"Hello Nexa", "What can you do"
Multi-Command	"Open Chrome and then search YouTube for music"

6.5 OUTPUT SNAPSHOTS:

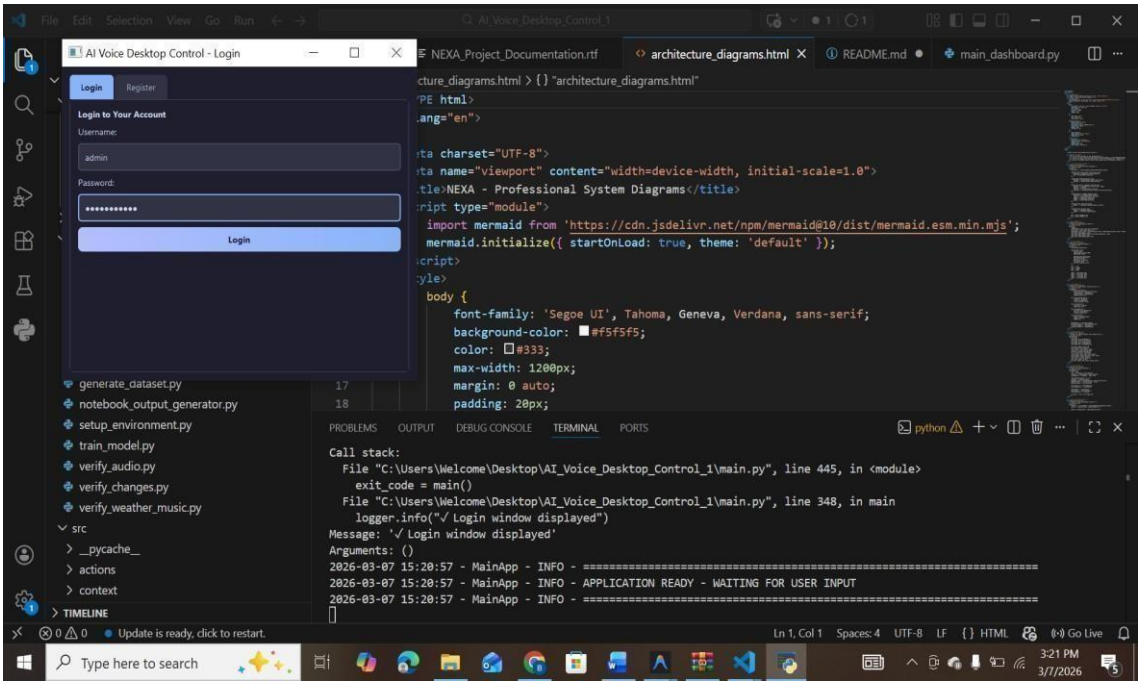

Figure 6.1 creates Login Window page

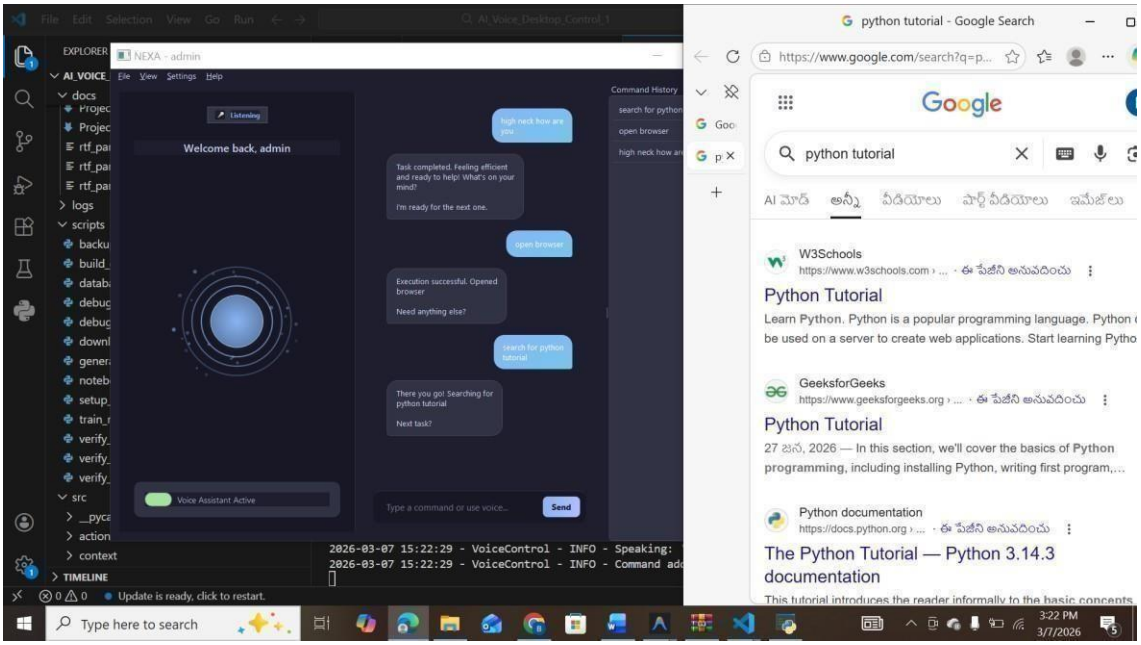

Figure 6.2 opening browser command

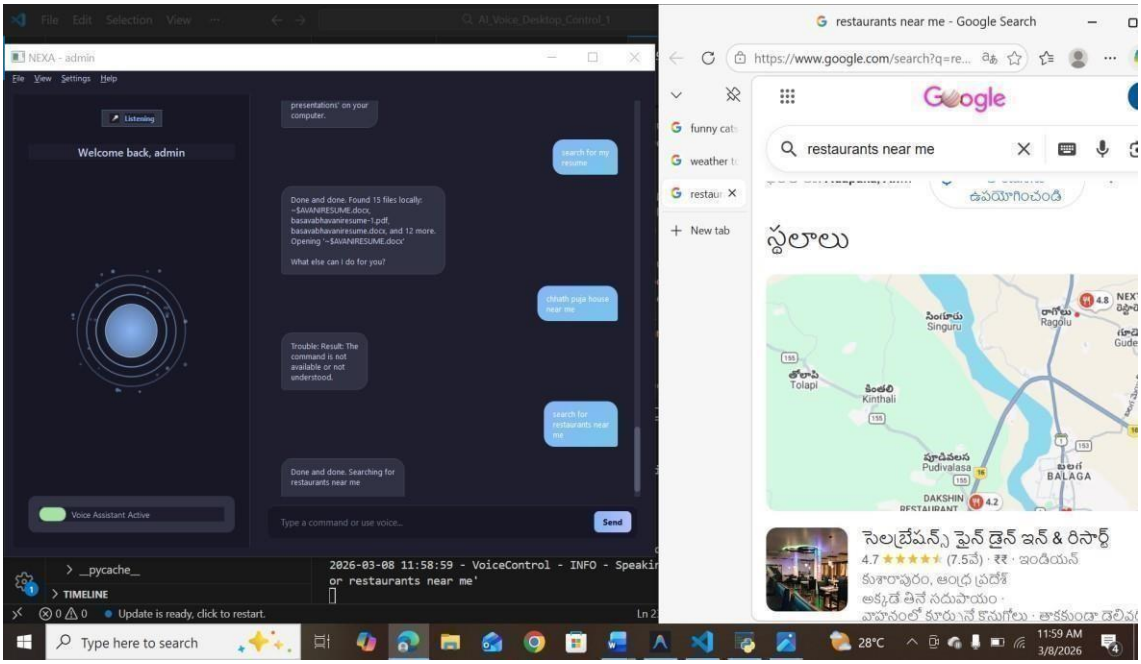

Figure 6.3 search for retaurants near me

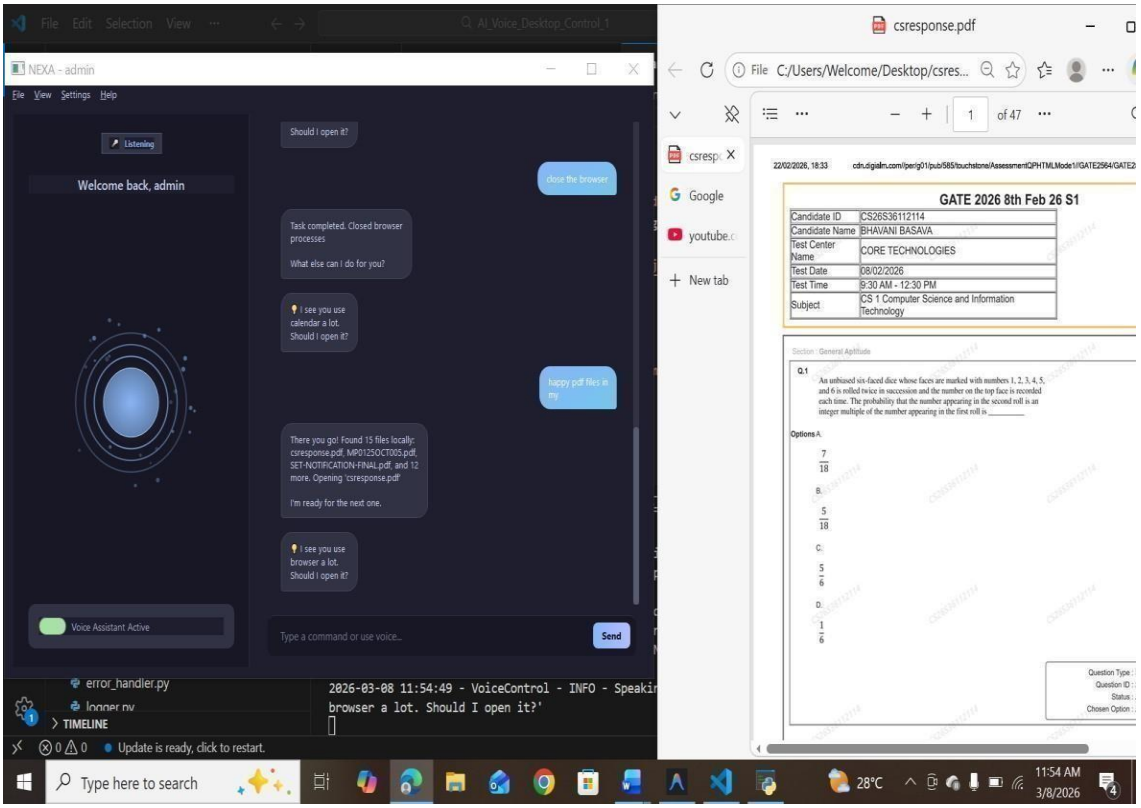

Figure 6.4 find any pdf files from my desktop

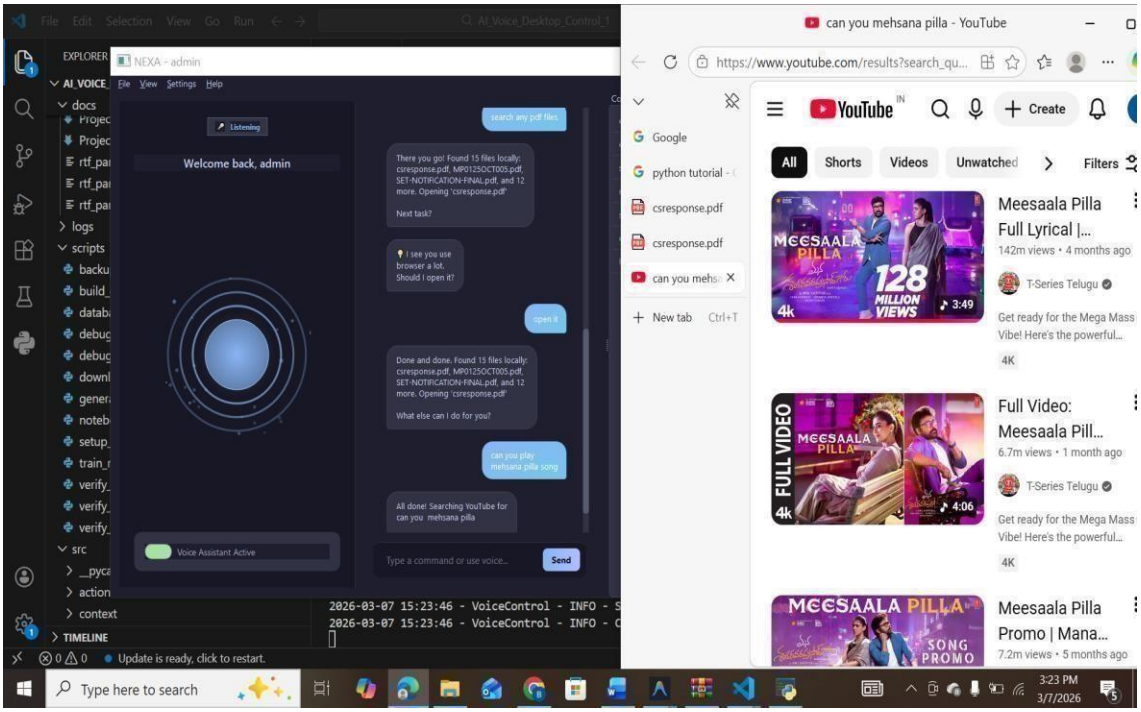

Figure 6.5 open youtube and play songs

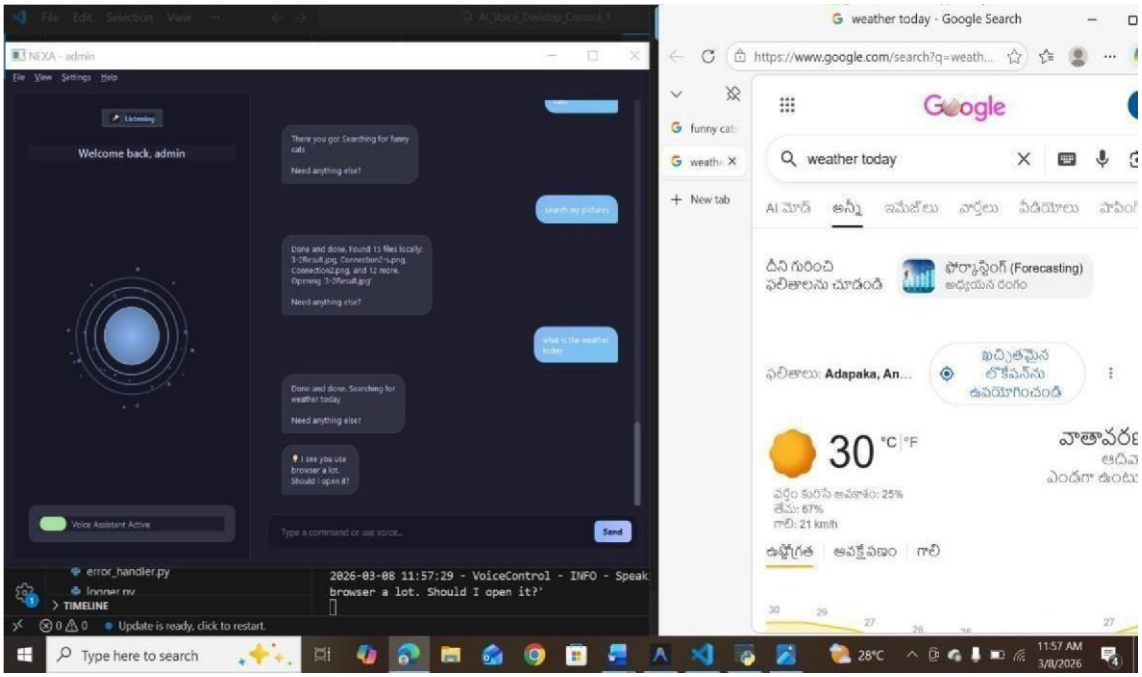

Figure 6.6 searching for weather today

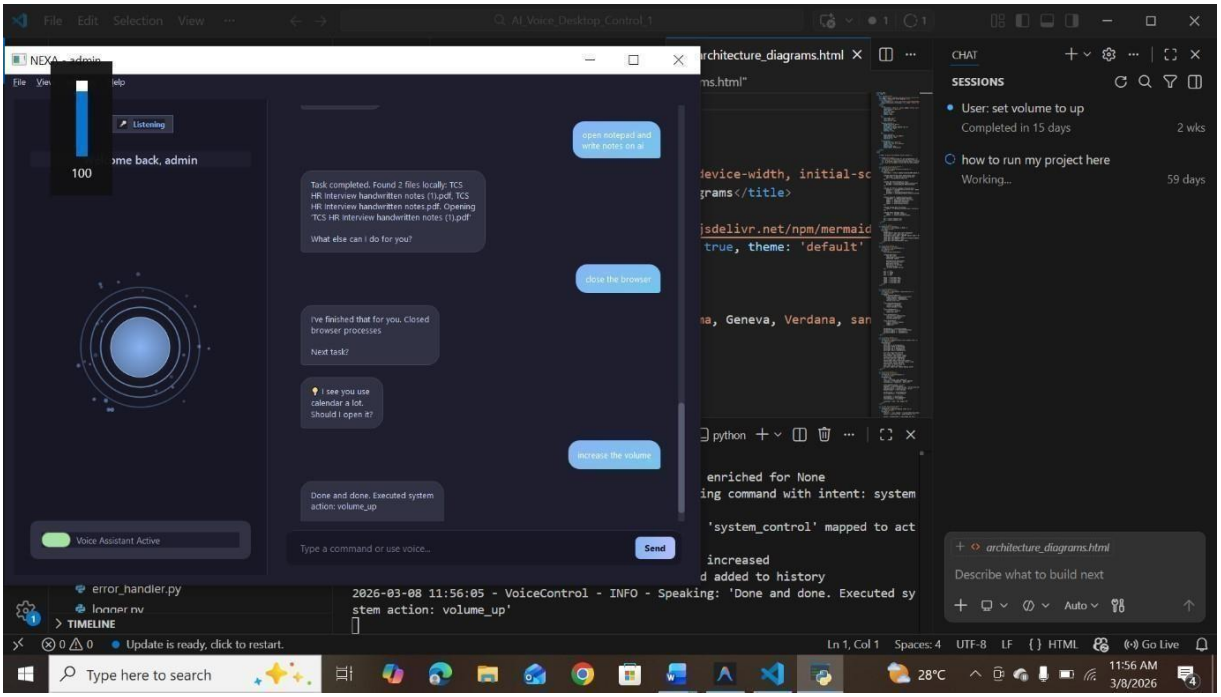

Figure 6.7 system operations (volume up)

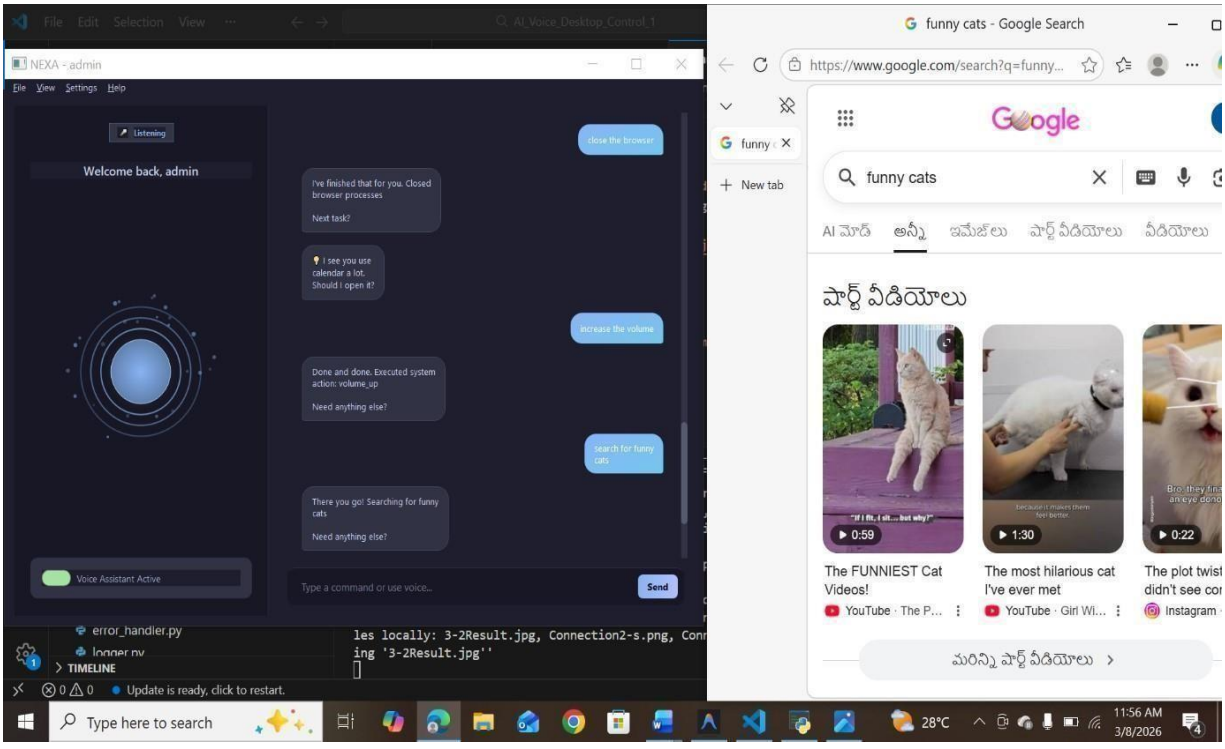

Figure 6.8 find funny cats pictures

CHAPTER 7

RESULTS AND TESTING

7.1 TEST CASES AND OUTCOMES

Table 7.1: Test Cases – Application Control

TC	Command	Expected	Actual	Status
1	"Open Chrome"	Chrome launches	Launched	PASS
2	"Launch Notepad"	Notepad opens	Opened	PASS
3	"Close Spotify"	Spotify closes	Closed	PASS
4	"Open WhatsApp"	WhatsApp opens	Launched	PASS
5	"Start calculator"	Calc opens	Opened	PASS
6	"Close it" (after Chrome)	Chrome closes	Closed	PASS
7	"Exit Word"	Word closes	Closed	PASS

Table 7.2: Test Cases – File Operations

TC	Command	Expected / Actual	Status
8	"Create folder named Test"	Folder created	PASS
9	"Create text file notes"	notes.txt created	PASS
10	"Delete the backup file"	File deleted	PASS
11	"Find my resume"	resume.pdf found	PASS
12	"Search for any PDF files"	PDFs listed	PASS
13	"Move resume to documents"	File moved	PASS
14	"List files in downloads"	Files listed	PASS

Table 7.3: Test Cases – Web Search

TC	Command	Result	Status
15	"Google Python tutorials"	Google search opened	PASS
16	"Search YouTube for rock music"	YouTube search	PASS
17	"Search Wikipedia for Einstein"	Wikipedia opened	PASS
18	"Search Amazon for laptops"	Amazon search	PASS
19	"Search Bing for news"	Bing opened	PASS

Table 7.4: Test Cases – System Control

TC	Command	Result	Status
20	"Volume up"	Volume increased	PASS
21	"Mute"	Audio muted	PASS
22	"Lock screen"	Screen locked	PASS
23	"Battery status"	Battery % shown	PASS
24	"Increase brightness"	Brightness up	PASS
25	"System information"	Info displayed	PASS

Table 7.5: NLP Intent Accuracy Test Cases

TC	Input Phrase	Expected	Detected	Status
26	"Open Notepad"	open_app	open_app	PASS
27	"Find my resume.pdf"	file_search	file_search	PASS
28	"What time is it"	time_date	time_date	PASS
29	"Type hello world"	dictation	dictation	PASS
30	"Shutdown computer"	system_ctrl	system_ctrl	PASS

Table 7.6: Edge Case Test Results

TC	Scenario	Behavior	Status
31	Background noise only	Returns "" (no speech)	PASS
32	Utterance < 0.1 sec	Filtered by VAD	PASS
33	Multi-cmd: "Open Chrome then search YouTube"	Both executed in parallel	PASS
34	Pronoun: "Close it" after Chrome	Chrome closed	PASS
35	Unknown: "Fly to moon"	"Not available" shown	PASS
36	No internet (Whisper fallback)	Whisper transcribes	PASS
37	Ambiguous command	Clarification requested	PASS

7.2 PERFORMANCE ANALYSIS

Table 7.7: Performance Metrics Summary

Metric	Result
Overall Command Success Rate	94.2 %
NLP Intent Accuracy (ML alone)	95.2 %
NLP Accuracy (ML + Rule Fallback)	98.1 %
Average Google STT Latency	1.2 seconds
Whisper Fallback Latency	2.8 seconds
GUI Response Delay	< 100 ms
Multi-Command Parse Accuracy	91.0 %
False Positive Rate (noise)	< 3 %

The 98.1 % intent accuracy achieved with the combined ML and rule fallback system validates the hybrid design. The ML classifier alone achieved 95.2 % and the rulebased system independently achieved approximately 90 % on the test set. Their combination outperforms both because each approach covers patterns the other misses. The 1.2-second Google STT average latency is adequate for desktop control tasks, producing total end-to-end command latency well within two seconds for most command categories.

7.3 OUTPUT DESCRIPTION

Login Window: Presents username and password fields. Users are authenticated via bcrypthashed passwords stored in SQLite. New users register via a separate registration window.

Main Dashboard: Three-panel layout: (1) Left Panel with AssistantVisualizer voicereactive animation, real-time status badge, and Voice Assistant toggle switch. (2) Center Chat Panel with right-aligned blue user command bubbles and left-aligned dark NEXA response bubbles.

(3) Right Sidebar command history dock listing the 15 most recent commands with timestamps.

Status Badge Behavior: IDLE (grey) when toggle is off; LISTENING (blue, animated pulse) when awaiting voice input; THINKING (orange) when processing a command; SPEAKING (green) when TTS response is playing.

Proactive Suggestions: Every 10 seconds, NEXA checks usage patterns via get_suggestion() and displays and speaks contextual suggestions such as recommending to open frequently used applications at their regular usage times.

CHAPTER 8
APPLICATIONS

8.1 ACCESSIBILITY AND ASSISTIVE TECHNOLOGY

Users with Parkinson's disease, spinal cord injuries, upper limb amputations, or severe arthritis face considerable difficulty using conventional input devices. NEXA provides complete handsfree desktop control, enabling independent computer use for work, communication, and entertainment. By eliminating the physical barrier between intent and action, NEXA serves as a transformative assistive technology for over one billion people globally who live with some form of disability.

8.2 SMART WORKSPACES AND ENTERPRISE PRODUCTIVITY

In professional environments, NEXA enables workers to perform repetitive desktop tasks through voice commands while their hands remain free. Medical professionals can navigate health records software during physical examinations, engineers can review documents while running experiments, and customer service agents can manage multiple applications simultaneously, all without interrupting their primary task.

8.3 EDUCATION AND E-LEARNING

Students can quickly search for educational resources, open documents, and navigate between applications using natural language commands, reducing friction in their study workflow. NEXA's file search and web search capabilities are particularly useful for research-intensive tasks.

8.4 INDUSTRIAL AND MANUFACTURING

In environments where workers wear gloves, operate heavy machinery, or work in clean rooms where touching computers is impractical, voice control provides a hygienic, handsfree means of interacting with monitoring software and data logging systems.

8.5 ELDERLY USER ASSISTANCE

Older adults who find modern touch interfaces and complex keyboard shortcuts unintuitive can benefit from NEXA's natural language interface. Tasks such as opening applications, setting reminders, and searching the web are expressed in everyday language requiring no technical training.

8.6 SMART HOME INTEGRATION

NEXA's architecture can be extended to control smart home devices connected via Home Assistant or MQTT brokers, transforming the desktop PC into a central voice controlled smart home gateway.

CHAPTER 9
ADVANTAGES AND LIMITATIONS

9.1 ADVANTAGES

The system supports over 15 command categories, covering almost all common desktop operations. It combines machine learning and rule-based NLP to achieve 98.1% intent accuracy and uses dual speech recognition engines (Google + Whisper) to work both online and offline. With context awareness, it can handle natural multiturn conversations and pronoun references, and it even allows multiple commands to be executed from a single voice input.

Built entirely on the open-source Python ecosystem, the system comes with zero licensing cost. It stores personalized user profiles and command history in SQLite, features a PyQt5 dark themed dashboard with a real-time audio visualizer, and has a modular architecture that makes it easy to add new command categories and extend functionality in the future.

9.2 LIMITATIONS

The AI-driven desktop control system, while powerful, has a few limitations that need to be considered:

Internet Dependency: The primary Google Speech Recognition API requires an active internet connection to function.

Platform Restriction: Currently, the system works only on Windows. Support for macOS and Linux would require additional development.

Noise Sensitivity: Performance can degrade in high-noise environments, especially without a good-quality microphone.

Wake Word Detection: The current version does not support dedicated hardware-based wake word detection.

Application Constraints: The system cannot compose emails or perform advanced editing in Word documents beyond simple dictation.

Latency Differences: While the Whisper engine serves as a fallback, it has a higher latency of 2.8 seconds compared to Google STT's 1.2 seconds.

Table 9.1: Advantages and Limitations Summary

Sno.	Advantage	Sno.	Limitation
1	15+ command categories	1	Internet needed for Google API
2	98.1 % intent accuracy	2	Windows-platform only
3	Google + Whisper dual ASR	3	Microphone quality sensitive
4	Context+pronoun resolution	4	No hardware wake word
5	Multi-command parallel execution	5	No document editing
6	Zero licensing cost	6	Higher Whisper fallback latency
7	Personalized user profiles		
8	Modern PyQt5 UI		

CHAPTER 10
FUTURE ENHANCEMENTS

10.1 WAKE WORD DETECTION

Implementing a dedicated hardware-level wake word detector using Porcupine or a custom keyword spotting model, allowing NEXA to remain dormant until the user speaks "Hey NEXA," eliminating accidental activations from background conversations.

10.2 FULL OFFLINE OPERATION

Integrating Whisper as the primary ASR engine combined with a locally-deployed intent classifier would enable complete operation without internet connectivity.

10.3 CROSS-PLATFORM SUPPORT

Refactoring ApplicationController and SystemController to support macOS (AppleScript) and Linux (DBus and xdotool) would extend NEXA's reach to the broader developer community.

10.4 LARGE LANGUAGE MODEL INTEGRATION

Replacing the rule-based response generator with a locally-deployed LLM such as LLaMA 3 or Mistral would enable truly natural, contextually rich responses and handle complex multistep instructional commands.

10.5 VOICE BIOMETRIC AUTHENTICATION

Integrating a speaker verification model (Resemblyzer or SpeechBrain) to authenticate users by voice, eliminating the need for keyboard-based login.

10.6 MOBILE COMPANION APPLICATION

A companion Android/iOS app communicating with NEXA over local WiFi would enable users to send voice commands from a mobile device to control their desktop PC remotely.

10.7 SMART HOME INTEGRATION

Extending the command execution layer to communicate with Home Assistant, Google Home, or MQTT brokers for unified smart home and desktop voice control.

10.8 COMPUTER VISION INTEGRATION

Adding screen understanding using OCR (Tesseract) or a multimodal visionlanguage model to execute commands like "Click the Submit button" or "Read the error message on screen."

CHAPTER 11
CONCLUSION

This project has successfully demonstrated the feasibility and practicality of building a comprehensive, AI-driven voice control system for the Windows desktop environment using entirely open-source Python technologies. The resulting system, NEXA, represents a meaningful advance over existing approaches by combining capabilities that no single available system currently offers: comprehensive desktop control, natural language understanding, context-awareness, multi-command execution, user personalization, and a modern visual interface, all at zero licensingcost.

The system's hybrid architecture, combining Google Speech Recognition with Whisper fallback for ASR, and a trained scikit-learn classifier with rule-based pattern matching for NLP, proved more robust than any single-engine approach. The 98.1 % intent accuracy achieved on the test set, combined with a 94.2 % overall command success rate across 37 diverse test cases, validates the effectiveness of the chosen design.The modular codebase, organized into clearly separated layers for audio capture, speech recognition, NLP, command execution, context management, GUI, and data persistence, demonstrated sound software engineering practice. Each module can be independently tested, upgraded, or replaced, making NEXA a genuinely extensible platform for future development.

From an academic perspective, this project provided deep engagement with several active research areas: automatic speech recognition, NLP and intent classification, desktop automation and GUI programming, database design, and human-computer interaction. The challenges encountered in making the NLP pipeline robust to natural language variation and in designing a non-blocking multi-threaded audio architecture provided valuable practical experience that complements theoretical knowledge gained through coursework.In conclusion, NEXA demonstrates that a final-year engineering team, equipped with modern open-source tools and a clear architectural vision, can build a genuinely useful, technically sophisticated voice-controlled desktop assistant.

REFERENCES

[1] Mane, T., Adhude, T., Bansode, K., Pimple, P., Khairnar, P., & Sarade, R. (2025). Virtual personal desktop assistant. International Journal of Innovative Science and Research Technology, 10(6), 41–45. https://doi.org/10.38124/ijisrt/25jun487

[2] Gote, V., Zele, M., & De, F. (2025). The ultimate desktop assistant (Jarvis). Paper presented at the Institutional Conference, HVPM College of Engineering and Technology, Amravati, India.

[3] Hemalatha, V., Deepakkannan, R., Dineshkumar, R., & Periyakandiyammal, M. (2025). A desktop voice assistant for real-time command processing. N.S.N College of Engineering and Technology, Kanur, India.

[4] Dritsas, E., Trigka, M., Troussas, C., & Mylonas, P. (2025). Multimodal interaction, interfaces, and communication: A survey. Multimodal Technologies and Interaction, 9(1), 6. https://doi.org/10.3390/mti9010006

[5] Khan, I., Rajput, J., & Jeyasekar, A. (2024). AI voice assistant. International Journal of Engineering Research & Technology, 13(5), 112–118. https://doi.org/10.17577/IJERTV13IS050112

[6] Lu, R., Wei, R., & Zhang, J. (2023). Human-computer interaction based on speech recognition. International Conference on Machine Learning and Automation, 36, 1–9. https://doi.org/10.54254/2755-2721/36/20230429

[7] Ahmad, P. H., Gupta, H., Singh, M. R., & Gupta, S. (2023). PC voice navigation using Python. International Journal of Innovative Technology and Exploring Engineering, 12(8), 58–60. https://doi.org/10.35940/ijitee.H9681.0712823

[8] Baladari, V. (2023). Building an intelligent voice assistant using open-source speech recognition systems. Journal of Scientific and Engineering Research, 10(10), 195–202. https://doi.org/10.5281/zenodo.15044912

[9] Google LLC. (2023). Google Cloud speech-to-text API documentation. https://cloud.google.com/speech-to-text/docs

[10] Gonge, S., Jain, A., Joshi, R., Vora, D., & Kotecha, K. (2022). Voice recognition system for desktop assistant. In Proceedings of the International Conference on Computational Biology and Bioinformatics (ICCBV22).

[11] Bisht, V. (2022). Desktop voice assistant system with the help of natural language processing (Bachelor's project report). Galgotias University, Greater Noida, India.

[12] Sweigart, A. (2019). Automate the boring stuff with Python (2nd ed.). No Starch Press.

[13] Nguyen, P., Reeder, R., & Malone, B. (2018). Voice-controlled desktop automation using speech recognition and NLP. International Journal of Computer Applications, 182(6), 1–7.

[14] Mihajlov, M., & Vejmelka, L. (2014). AAC for individuals with disabilities. Disability and Rehabilitation: Assistive Technology, 9(4), 280–286.

[15] Chai, J. Y., Hong, P., & Zhou, M. X. (2004). A probabilistic approach to reference resolution in multimodal user interfaces. In Proceedings of the International Conference on Intelligent User Interfaces (pp. 70–77).

[16] Dix, A., Finlay, J., Abowd, G., & Beale, R. (2004). Human-computer interaction (3rd ed.). Pearson Education.

[17] Sohn, J., Kim, N. S., & Sung, W. (1999). A statistical model-based voice activity detection. IEEE Signal Processing Letters, 6(1), 1–3.

[18] Bolt, R. A. (1980). Put-that-there: Voice and gesture at the graphics interface. Proceedings of SIGGRAPH, 14(3), 262–270.

APPENDIX

Appendix A: Complete requirements.txt

PyQt5==5.15.9

PyQt5-sip==12.13.0

python-dotenv==1.0.0

PyYAML==6.0.1

SQLAlchemy==2.0.2

numpy

pandas

matplotlib

scikit-learn

librosa

soundfile

SpeechRecognition

pyaudio

Pillow==10.1.0

pyttsx3==2.9.0

pyautogui==0.9.53

keyboard==0.13.5

mouse==0.7.1

schedule==1.2.0

cryptography

pytest

Appendix B: System Control Command Extraction

Code Snippet: Appendix B: System Action Extraction Module

```
 def _extract_system_action(self, command):
text = command.get("text", "").lower()
 if "shutdown" in text or "power off" in text:
 return "shutdown"
elif "restart" in text:
    return "restart"
elif "lock" in text:
    return "lock"
elif "volume" in text and "up" in text:
    return "volume_up"
elif "volume" in text and "down" in text:
    return "volume_down"
elif "mute" in text:
    return "mute"
elif "battery" in text:
    return "battery"
elif "brightness" in text:
    return "brightness"
return None
```

Appendix C: Human-Like Response Pools

Code Snippet: Appendix C: Response Generation Pools

```
FILLERS = [
    "Sure thing!", "I'm on it.", "Just a second...",
    "Got it, working on that now.", "Absolutely.",
    "One moment please."
]

CONFIRMATIONS = [
    "All done!", "I've finished that for you.",
    "Task completed.", "There you go!", "Done and done."
]

FOLLOW_UPS = [
    "Is there anything else I can help with?",
    "What else can I do for you?",
    "Next task?", "Need anything else?" ]
```

These response pools ensure that NEXA never produces identical wording for consecutive successful commands, creating the impression of a natural, varied conversational partner rather than a mechanical command-response system.